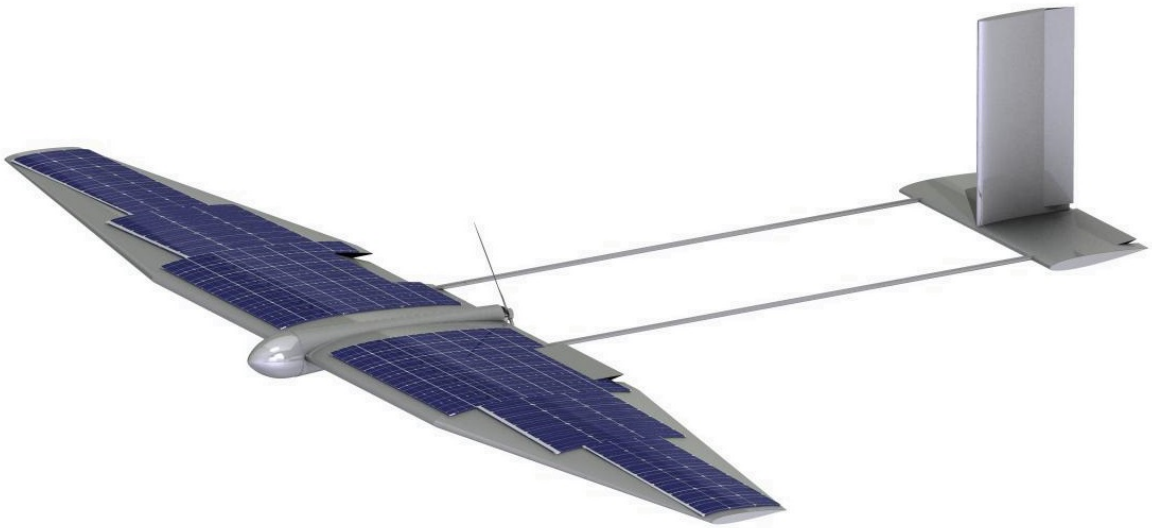
Imperial College London

Year 3 Group Design Project

Solar Challenge 2016

Aerodynamics Team



Academic Supervisor
Dr Yongyun Hwang

Student
Lionel Kloetzli

CID number
00866772

Submission Date
13/06/2016

Department
Aeronautics

Course
MEng in Aeronautical Engineering

Module
AE3-306 Group Design Project

Academic year
2015-2016

Abstract

The aim of the Solar Challenge project was to design a solar UAV, the *Solar Flare*, capable of performing competitively in the race. An additional application was chosen, including marine tracking and site monitoring, which presented very different operating conditions. The aim of the aerodynamics team was to achieve a design capable of maximising the race performance while allowing for a high endurance, low speed application. An elliptically loaded 2.65 m span, 1 m^2 wing with an aspect ratio of 7 was designed to carry the 3 kg *Solar Flare* with several considerations, including solar power extraction efficiency and ease of manufacture. Numerous analytical and computational methods were first validated to then evaluate the aerodynamic performance of the wing and of the full UAV. Consistent and reliable results were obtained, confirming the feasibility of such a design over both operating conditions. Indeed, at a cruise velocity of 10 m/s, determined for the application, a lift-to-drag ratio of 20 was obtained, compared to 12.7 for the higher-speeds (15 to 20 m/s) race conditions. Collaboration with the other design teams was essential in achieving a successful design.

Contents

List of Figures

List of Tables

Nomenclature

$\boldsymbol{\alpha}$ Angle of attack

$\boldsymbol{a_0}$ 2D lift curve slope

$\boldsymbol{\text{Æ}R}$ Aspect ratio

$\boldsymbol{b}$ Total span

$\boldsymbol{c}$ Chord

$\boldsymbol{C_{D_0}}$ Zero-lift drag coefficient

$\boldsymbol{C_{D_i}}$ Induced drag coefficient

$\boldsymbol{C_D}$ Drag coefficient

$\boldsymbol{C_d}$ Sectional drag coefficient

$\boldsymbol{C_f}$ Skin friction drag coefficient

$\boldsymbol{C_L}$ Lift coefficient

$\boldsymbol{C_{L_{max}}}$ Maximum lift coefficient

$\boldsymbol{C_l}$ Sectional lift coefficient

$\boldsymbol{C_{l_{max}}}$ Maximum 2D lift coefficient

$\boldsymbol{C_M}$ Pitching moment coefficient

$\boldsymbol{C_m}$ Sectional pitching moment coefficient

$\boldsymbol{C_P}$ Pressure coefficient

$\boldsymbol{\delta}$ Boundary layer thickness

$\boldsymbol{e}$ Oswald efficieny factor

$\boldsymbol{FF}$ Form factor

$\boldsymbol{g}$ Gravitational acceleration ($9.81\,m/s^2$)

$\boldsymbol{L/D}$ Lift-to-drag ratio

$\boldsymbol{LLT}$ Lifting Line Theory

$\boldsymbol{M}$ Mach number

$\boldsymbol{MAC}$ Mean aerodynamic chord

$\boldsymbol{Q}$ Interference factor

$\boldsymbol{Re}$ Reynolds number

$\boldsymbol{s}$ Semi-span

$\boldsymbol{S}$ Wing surface area

S_{wet} Wetted surface area

$\boldsymbol{\theta}$ Boundary layer momentum thickness

$\boldsymbol{U_e}$ Velocity at the edge of the boundary layer

$\boldsymbol{U_\infty}$ Free stream velocity

$\boldsymbol{V}$ Operating Velocity

$\boldsymbol{V_{stall}}$ Stall velocity

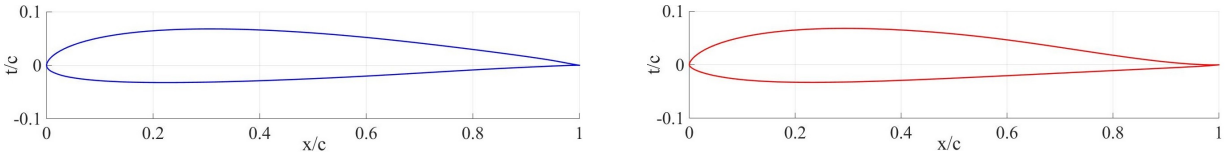
$\boldsymbol{VG}$ Vortex generator

1 Introduction

The aim of this project is to design a hand launched fixed wing solar UAV under 5 kg in mass, with a maximum wing loading of 7.5 kg/m^2, a maximum wing area of 1.5 m^2, and flying at low altitudes (sea level conditions) [1] [2]. The *Solar Flare* is made to be competitive in the *Solar Challenge* race as it is optimised for a range of operating velocities. Furthermore, it also had to perform in the application conditions (marine tracking and site monitoring) for which, contrastingly, endurance had to be maximized, which entails for instance a lower velocity and Reynolds number. The wing was therefore designed for maximum performance during the race but by also taking into account the range of operating velocities - between 10 m/s for the application and 16 m/s for the average race velocity. One of the main challenges was that the power available is limited by the amount of solar energy available. As a result, the thrust being limited, the drag had to be minimized. This report highlights the design process by going through the challenges and suggested solutions over the two iterations achieved and covers initial trade-off analyses and performance estimates before presenting the aerofoil selection process, the wing design, and lift and drag evaluation. Finally, a study on aerodynamic enhancement devices will be shown with the objective of improving the overall performance for both the race and the application.

2 Tailless Aircraft Feasibility Study

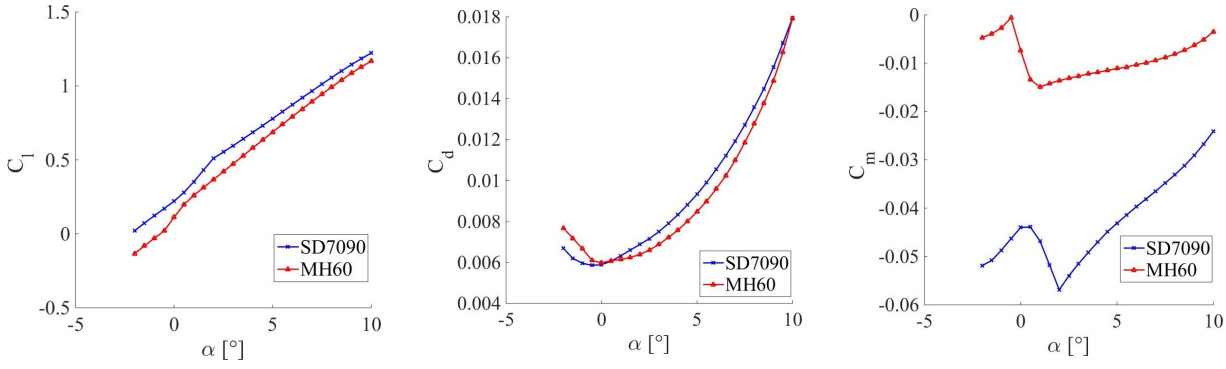
Early on in the project, a flying wing configuration was being considered from the ones shortlisted by FATT01. Indeed, from an aerodynamics point of view, as this removes the need for additional lifting surfaces, the wetted area becomes smaller, thus leading to a smaller skin friction drag. However, a tailless aircraft is more complex to design as stability becomes an issue that can be dealt with by using sweep to distribute the lift such as to achieve longitudinal stability. A very small pitching moment coefficient is also preferable and can be obtained with reflexed aerofoils. These features however lead to a reduced aerodynamic performance of the wing.
To compare both configurations, a Vortex Lattice Method on XFLR5 (a 3D derivative of XFoil) with a viscous analysis that extrapolates the 2D viscous drag values from XFoil to a 3D viscous value was used. This gives very rough estimates that were merely used to compare both configurations. Moreover, as the configurations were tested at very low angles of attack (far from stall), the analysis was considered valid to the accuracy required at this early design stage [4]. The same aspect ratio, wing area and overall lift were considered for both the conventional configuration with tail and fuselage, and a flying wing with 25° sweep (Refer to Appendix A for the geometries). The drag coefficient was found to be 30% larger for the tailless configuration ($C_D = 0.009$ for the flying wing compared to $C_D = 0.007$ for the conventional configuration). This was due to the tailless configuration flying at a higher angle of attack for the same overall lift and due to the reflexed aerofoil used which was less aerodynamically efficient. Indeed, an initial brief aerofoil selection, for similar thickness-to-chord ratios, lead to the MH60 (reflexed aerofoil with good performance at low Reynolds numbers [5]) being chosen for the flying wing and to the SD7090 [6] for the conventional configuration:



(a) SD7090 **(b)** MH60

Figure 2.1: Aerofoils chosen to compare a flying wing and a conventional configuration

The lift, drag and moment polars, obtained from XFoil, are presented in Figure 2.2.



(a) Lift polar **(b)** Drag polar **(c)** Pitching moment polar

Figure 2.2: 2D polars for the two aerofoils considered

The slight reflex on the MH60 yields indeed a pitching moment coefficient almost equal to zero - approximately 75% smaller than for the SD7090 - but has a 26% lower lift even if the drag coefficient values are very similar. As a result, the overall performance of a tailless design is decreased in comparison to a conventional configuration and this study, even though relatively inaccurate, was deemed sufficient to disregard the flying wing configuration from an aerodynamic point of view. A conventional configuration was thus chosen and the following results all assume such a design.

3 Initial Performance Estimates

To be able to design the wing of the UAV, on which many other FATTs depended on, the aerodynamics team required initially a design velocity and a wing area to start the first iteration for which it was chosen to maximize cruise velocity. Drag had to be first estimated to find the thrust required and for FATT02 to estimate the power needed to sustain level flight. The drag was modeled in the following way:

$$D = \frac{1}{2}\,\rho\, V^2\, S\,(C_{D_0} + C_{D_i}) \tag{3.1}$$

where C_{D_0} was estimated from [3] using the equivalent skin friction method:

$$C_{D_0} = C_{f_e}\,\frac{S_{wet}}{S} \tag{3.2}$$

taking $C_{f_e} = 0.0055$ from [3] (typical value for light aircraft with a single engine). S is the wing area and S_{wet} was estimated using a simple formula in [3]. The induced drag coefficient was obtained as:

$$C_{D_i} = \frac{C_L^2}{\pi\, \mathcal{AR}\, e} \quad \text{and} \quad C_L = \frac{2\,m\,g}{\rho\, V^2\, S} \tag{3.3}$$

The Oswald efficiency factor, e, was determined from a formula in [3] depending solely on $\mathcal{AR}$ which was fixed at $\mathcal{AR} = 9$ for the first iteration, as an initial compromise between aerodynamic efficiency and added structural weight [13]. The drag is therefore dependent on only two unknowns which are the wing surface area and the velocity. S was allowed to vary from the minimum allowable from the wing loading constraint, assuming a mass of $m = 3.5\,kg$ (given by FATT01 as an initial estimate [14]), to the maximum wing area of $1.5\,m^2$ while varying velocity as well. The power ratio, defined as the power available divided by the power required, was then obtained by FATT02 [15] and resulted in a maximum achievable velocity of $25\,m/s$. Furthermore, the maximum wing surface area yielded the maximum power ratio, as presented in Figure 3.1, and was thus used for the first iteration.

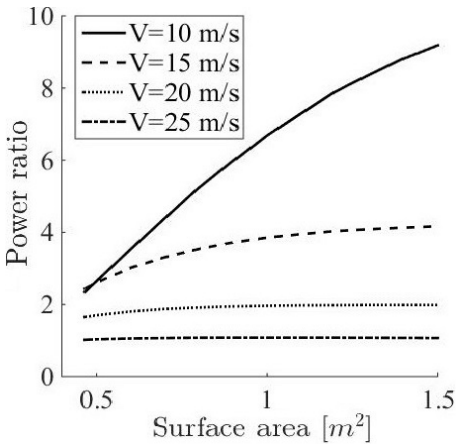


Figure 3.1: Power ratio curves over a range of velocities

For the 2^{nd} iteration, a MATLAB code (shown in Appendix B) was written in collaboration with FATT01 [14] and FATT02 [15] to evaluate the optimum performance of the UAV during the required figure-of-eight flight path for the race. Using values obtained from the first iteration and breaking down the race into acceleration, cruise, deceleration and turn segments, the minimum time to complete the circuit was found to be for the velocity profile shown in Figure 3.2. The race velocity for this iteration was thus determined as the average velocity during the race, $V = 16m/s$.

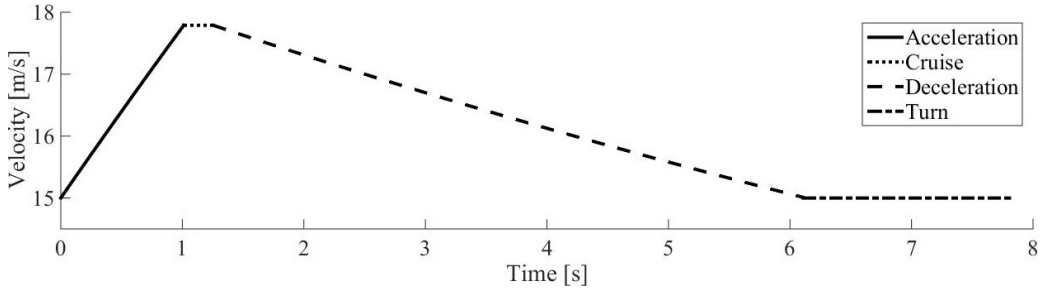


Figure 3.2: Velocity profile over half of the race circuit

The sizing of the wing was done differently for the second iteration as tip deflection and weight were found to be severe issues at the end of the first one. As a result, the wing surface area and the aspect ratio were iterated

upon with FATT04 [16] to achieve the required wing weight set by FATT01. This resulted in a decrease of the aspect ratio down to $\text{Æ}R = 7$ to reduce the wing span, thus minimising tip deflection. The wing area was also decreased to $S = 1\,m^2$ in an effort to keep the weight down and was deemed sufficient for propulsion purposes by FATT02 [15]. The wing was thus finally designed according to the following parameters:

Parameters	Mass (kg)	Wing Area (m^2)	$\text{Æ}R$	Design Velocity (m/s)	Design Lift Coefficient
Iteration 1	3.5	1.5	9	25	0.060
Iteration 2	3.0	1.0	7	16	0.19

Table 3.1: Design Parameters for Both Iterations

4 Aerofoil Selection

4.1 Theoretical Background and Operating Conditions

The *Solar Challenge* operates at low Reynolds numbers, between 180'000 at the tip and 490'000 at the root (values for the second iteration). This means that the flow, initially laminar and very sensitive to any disturbances or roughness, is very likely to separate on the upper surface of the aerofoil where it encounters rapidly an adverse pressure gradient [7]. Depending on the Reynolds number, which varies with span position, the laminar separated flow region usually transitions to turbulent and in some cases, might even reattach by bringing high-energy flow to the boundary layer [7]. The region of separated flow forms a laminar separation bubble which changes the effective shape of the aerofoil and thus reduces aerodynamic performance [7].
The aerofoil selection process was done by linking XFoil to MATLAB in order to find the most appropriate aerofoil out of a set of pre-selected ones considering an optimisation on several parameters. The MATLAB code used is presented in Appendix C. XFoil was deemed appropriate as it models rapidly all of the flow effects particular to this low Reynolds number range [8]. Indeed, XFoil uses a 2D inviscid panel method along with a viscous boundary layer formulation in the near-wall region to compute the flow field and it can also model transition and laminar separation bubbles using the e^N method by evaluating the growth of Tollmien-Schlichting waves [8]. N_{crit} was taken as 13 as it was found to be more accurate for Reynolds numbers greater than $200'000$ when a laminar separation bubble is present [9].

4.2 Design Process: Iteration 1

For the first iteration, the lift coefficient being very low ($C_L = 0.060$), there were no requirements for high lift at low angles of attack. However, it was essential to make sure that the UAV would fly at positive, albeit small, angles of attack. Therefore, camber was kept below 2%. An initial pre-selection of aerofoils with t/c between 10 and 12% was considered. A minimum of 10% was set together with FATT02 [15] and FATT04 [16] to fit components inside the wing and for structural reasons. The corresponding NACA 4- and 5-digit series were tested along with several low Reynolds number aerofoils found from the available literature [6] [7] [10]. All of these aerofoils would go through XFoil and the lift, drag and moment coefficients over a range of angles of attack would be generated at a Reynolds number of $Re = 600'000$ based on the mean chord. It was decided to maximise the 2D lift curve slope, a_0, and to minimize the drag and moment coefficients. Initially, it was thought with FATT05 [17] that minimizing the pitching moment coefficient would be beneficial for stability. Maximising a_0 was essential to minimise the angle of attack at which the UAV would be flying at for the application conditions. A merit function was created using normalized values where the drag and moment polars were integrated over a range of operating angles of attack to give a better estimate of the overall performance of the aerofoil. From these constraints, the NACA 0010 was selected due to the fact that its pitching moment coefficient was very small and that, as obtained later, the UAV would have needed to fly at negative angles of attack even with an aerofoil with 1% camber. It made physical sense that this aerofoil was selected because of the very low lift coefficient which did not allow for any lift maximisation. Therefore, the only way to increase the lift-to-drag ratio was to minimise drag.

4.3 Design Process: Iteration 2

Following the end of the first iteration, it was found that a very low pitching moment coefficient provided by the symmetric NACA 0010 did actually not have a positive impact on stability as the horizontal tail had to be large and far away from the wing [17], thus increasing structural weight. Hence the optimisation process for the second iteration was different and the merit function was revised. The objective was now to still maximize a_0 and minimize the integral of the drag polar, but this time, it was decided to maximise the integral of the pitching moment coefficient. Additionally, taking into consideration the application conditions, it was decided to maximise the integral of the lift-to-drag ratio (aerodynamic robustness) for optimum endurance and glide performance. The shortlisted aerofoils were the same as before with similar requirements on thickness and

camber still holding. These included for instance the Selig/Donovan SD6060, the Eppler E226 and the Quabeck HQ1.0/10 as well as the NACA 4- and 5-digit series. The following Table shows a summary of the conditions and inputs for the optimisation process for both iterations. The Reynolds number is based on the mean chord.

Parameters	Velocity (m/s)	Reynolds Number	Mach Number	Maximise	Minimise
Iteration 1	25	600'000	0.07	a_0	C_m & C_d
Iteration 2	16	410'000	0.05	a_0, C_l/C_d & C_m	C_d

Table 4.1: Summary of input parameters for the aerofoil selection process for both iterations

The NACA 1810, shown in Figure 4.1, was found to be the best compromise out of the requirements set for the 2^{nd} iteration and Figure 4.2 shows how it compares to the other tested aerofoils.

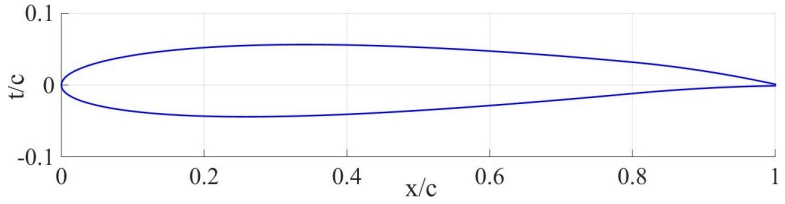
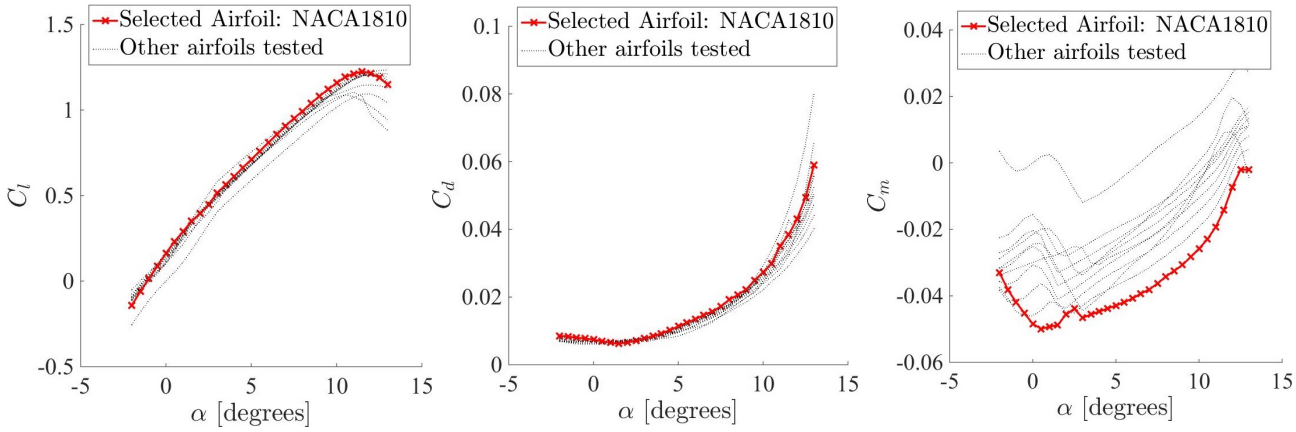


Figure 4.1: NACA 1810



(a) Lift polar **(b)** Drag polar **(c)** Moment coefficient polar

Figure 4.2: Aerofoil selection process results

This aerofoil had the following characteristics:

Lift curve slope $(/rads)$	$C_{l_{max}}$	Stall angle (degrees)	Zero-lift angle (degrees)
6.11	1.23	11.5	-1.10

Table 4.2: NACA 1810 aerodynamic characteristics for a Reynolds number of 410′000

The constraint on pitching moment coefficient, resulting in the maximum camber position being close to the trailing edge (80% chord), proved to be very beneficial as it enabled to reduce considerably the size of the tail [17] as well as to place it closer to the wing, thus allowing FATT04 to design a lower weight structure [16].

5 Wing Geometry Design Process

5.1 Design Philosophy

The previously explained aerofoil selection process was iterative on its own as it depended on the wing planform geometry, influencing the Reynolds number. The wing depends on the aerofoil as the 2D lift-curve slope is an input in the optimisation process for the planform geometry. Indeed, the wing geometry was obtained using Prandtl's Lifting Line Theory such that an optimum elliptical loading could be achieved as it gives the minimum induced drag. It was decided early on to go against having an elliptical chord distribution due to the manufacturing complexity which was deemed unreasonable by FATT01 and FATT04. Instead, the objective was to find the planform geometry that gave a lift distribution as close as possible to elliptical while staying relatively simple and easy to manufacture. Twist would then be used to achieve such a loading. However, a large twist meant that there would be some amount of losses in the solar energy hitting the solar panels.

Therefore, the idea was to find the planform geometry that minimised the twist required:

$$\alpha_{twist}(y) = \frac{C_L}{\pi \mathit{AR}} \left(\frac{8\,s}{a_0} \frac{\sqrt{1 - \left(\frac{y}{s}\right)^2}}{c(y)} \right) \tag{5.1}$$

Furthermore, as LLT is a linear theory that assumes small perturbations, minimising the twist and angles of attack would also make sure that the results obtained remain valid. Additionally, this is an inviscid, irrotational, incompressible and steady method. The most questionable assumption is inviscid flow as viscous effects can be quite large at these low Reynolds numbers as mentioned in the previous section. Compressibility is not an issue at such low speeds: the Mach number was only 0.05. Moreover, small perturbations meant that only a small amount of camber could be considered, which presented no issue as the NACA 1810 had only 1% of maximum camber. The aspect ratio and wing area being fixed, the twist distribution would be calculated for varying planform geometry, from a fully rectangular wing to a fully tapered wing, and including an initially rectangular section preceding a linearly tapered section. The spanwise position between the constant chord section and the tapered section was therefore varied along with the tip chord. The relative twist distribution would then be integrated and the lowest value would give the optimum wing geometry. The MATLAB code used for this optimisation process can be found in Appendix D.

5.2 Resulting Wing Geometry

The results for the 2^{nd} iteration are presented here as the process followed for the first iteration was the same, only differing by the input parameters values. Figure 5.1 shows the resulting twist distribution.

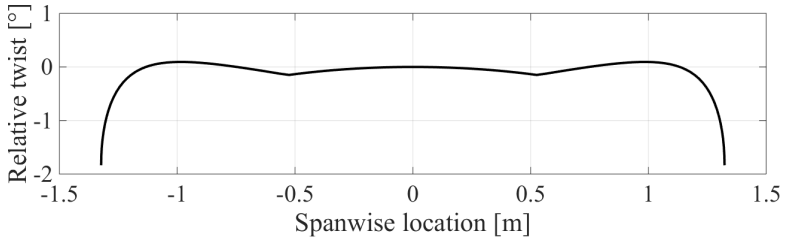


Figure 5.1: Relative twist distribution for the selected wing geometry

The final wing geometry is summarised in Table 5.1 and can be seen in Figure 5.2.

Span (m)	AR	Surface area (m^2)	Root chord (m)	Tip chord (m)	MAC (m)	Sweep angle (°)
2.65	7	1.0	0.47	0.17	0.38	0

Table 5.1: Main dimensions of the final wing design

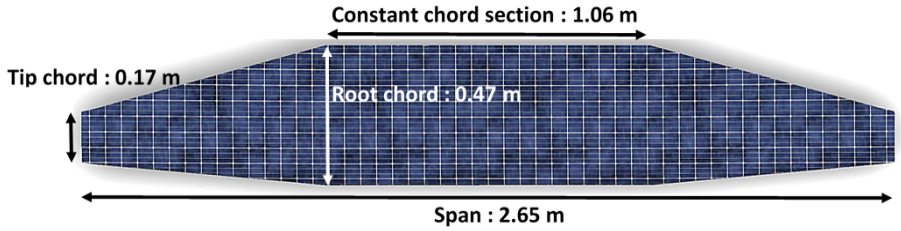


Figure 5.2: Geometry of the final wing design

Various inviscid and viscous analyses were then conducted to obtain reliable data on the wing's performance.

6 Wing Lift Estimation

6.1 Theoretical Background and Methodology

A collocation method was used as a first estimate to obtain the lift polar. However, this is based on the Lifting-Line Theory which makes a number of restrictive assumptions as seen previously. Moreover, it assumes only one bound vortex at the quarter chord which sheds vorticity downstream whereas having a distribution of bound vortices not only spanwise but also chordwise would be a closer approximation of the physics. A Vortex Lattice Method (also inviscid) was therefore used to compare results. Tornado was found to be an appropriate tool as it is based on MATLAB and enables to quickly evaluate the performance of an arbitrary wing geometry by discretising it in panels, each containing a horseshoe vortex [18]. As a result, the chordwise lift distribution can

also be modeled. However, the panels are placed on the camber line and LLT does not take into account the exact shape of the aerofoil as it assumes a flat plate. To be able to properly capture the effect of the wing's thickness, a 3D Panel method, in the form of the XFLR5 software, was used. The wing's surface is discretised in panels which consist of a constant strength singularity (source or sink) and the impermeability boundary condition is satisfied at the collocation points along with the required circulation to satisfy the Kutta condition [4]. VLM and Panel codes are both inviscid, incompressible, irrotational and steady: they are appropriate at small angles of attack as separation is not modeled [19]. Furthermore, they can only estimate induced drag but viscous drag is not calculated. While XFLR5 can extrapolate 2D viscous drag results from XFoil to 3D as mentioned in the tailless aircraft feasibility study, this method was deemed insufficiently accurate for drag estimation at this stage. These inviscid methods do however give good estimates for lift calculation. Tornado and XFLR5 were first validated against benchmark tests and experimental data to estimate the amount of uncertainty resulting from the use of such methods. These validation tests are presented in Appendix E and resulted in a maximum error of 3.4% for Tornado and of 7.2% for XFLR5, which validated the use of such softwares as long as they were used together and compared to LLT and CFD results. Finally, Computational Fluid Dynamics, CFD, with the Star-CCM+ software package allowed for viscous calculations of lift forces and, as will be seen later, of drag forces. The RANS equations were solved for steady, viscous, incompressible, and fully turbulent flow. An in-depth description of the physics models and of the mesh dependency is presented in [12].

6.2 Lift Polars and Results

The following 3D lift polars were obtained using the 4 methods aforementioned:

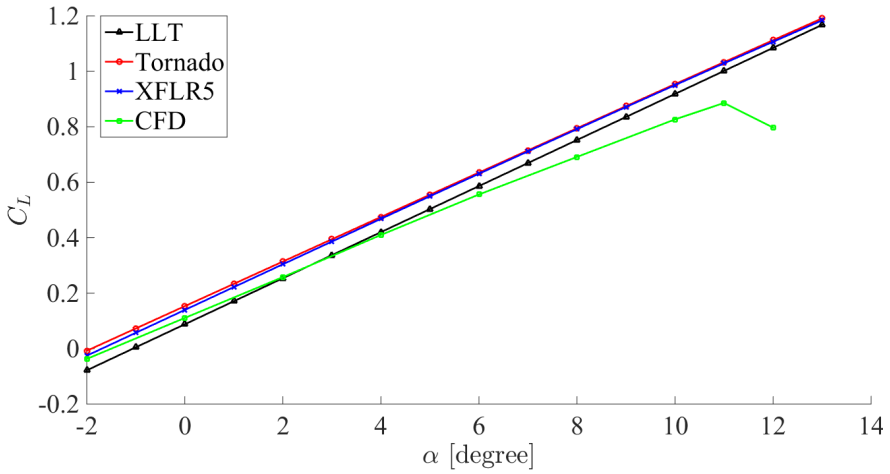


Figure 6.1: Lift polars for the final wing design

The VLM and the 3D Panel code produced very similar results (maximum 9% error) while LLT had the tendency to overestimate the lift curve slope. Out of the inviscid methods, the first two were taken as the most accurate due to a better representation of the wing geometry and lift distribution but LLT was nonetheless judged accurate enough to obtain the twist distribution required for an elliptical loading. Indeed, the spanwise lift distributions at the angle of attack required to produce the same overall lift as obtained from the inviscid methods were very similar (maximum error of 3%) as shown in Figure 6.2.

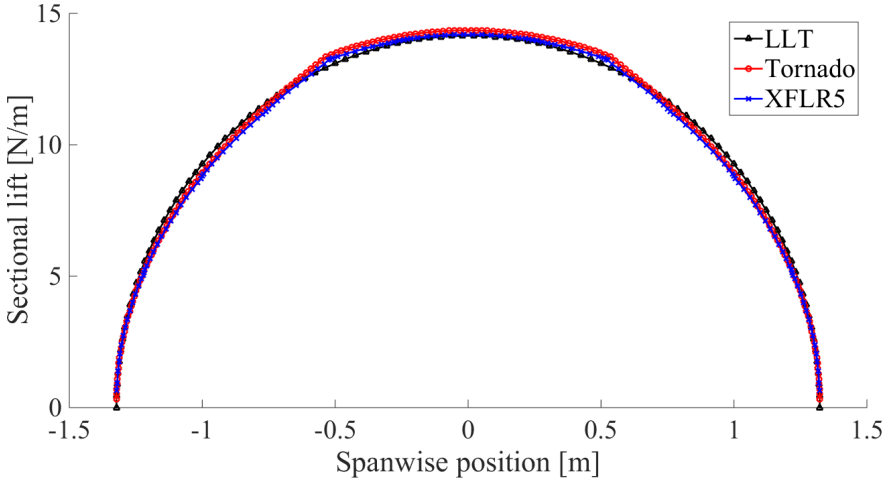


Figure 6.2: Spanwise lift distributions obtained from inviscid methods at design conditions for the race

Finally, the C_L values are very consistent at the low operating angles of attack (up to 6°) from all methods used. At larger angles of attack, the inviscid methods unsurprisingly predict a linear variation in C_L without stall. Contrastingly, the viscous (CFD) lift curve slope drops slightly before reaching a maximum C_L value, i.e. stall. This decrease in lift curve slope is due to the appearance of viscous-dominated separation regions over the wing. Overall, the main difference was that the angle of attack required to achieve the design lift coefficient differed for all the methods. This angle was important as it would determine the wing setting angle. A comparison of the main values from the lift polars is presented in the following table.

Method	Lift curve slope ($/rads$)	Zero-lift angle of attack (°)	Required setting angle (°)	$C_{L_{max}}$
LLT	4.76	-1.05	1.20	-
VLM	4.58	-1.90	0.43	-
Panel code	4.63	-1.69	0.59	-
CFD	4.09	-1.50	1.05	0.89

Table 6.1: Lift performance obtained from various methods for the second iteration wing

The final performance of the wing could then be determined from the previous results. It was decided to use the setting angle obtained from the CFD analysis. It was deemed the most accurate of the methods used due to less restrictive assumptions. Similarly, the conditions for the application could also be obtained. The stall velocity, estimated using $C_{L_{max}}$, was found to be $V_{stall} = 7.3\,m/s$ and an application velocity of $10\,m/s$ was decided upon with FATT01 [14] as optimum for the related mission profile. The main performance parameters for the race and the application are summarised in the following Table:

	Lift coefficient	Angle of attack (°)	Velocity (m/s)
Race	0.19	1.05	16
Application	0.48	5.0	10

Table 6.2: Summary of lift performance for race and application conditions

7 Drag Analysis

7.1 Wing Drag Estimation

Similarly to the previously shown lift polars, the drag coefficients were generated from the different methods and are shown in Figure 7.1, as a plot against C_L.

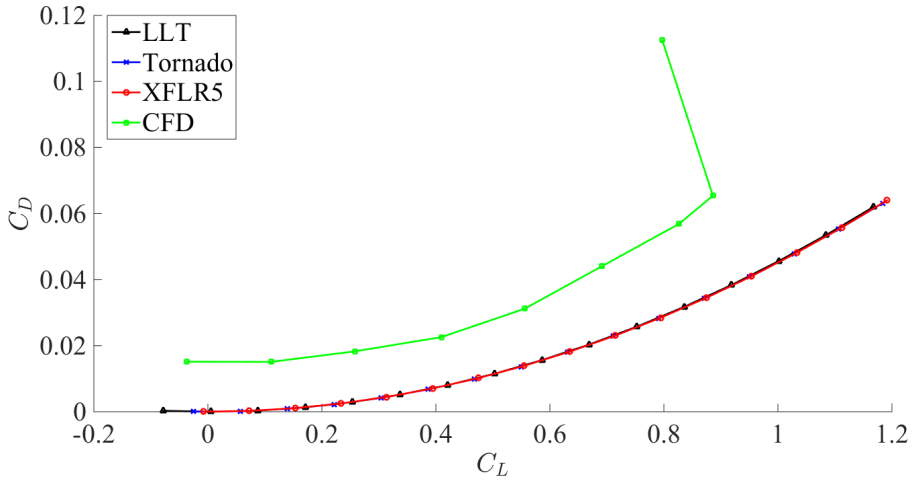


Figure 7.1: Drag polars for the final wing design

Evidently, a large difference in values between the inviscid methods and the viscous CFD analysis was observed. Indeed, the drag coefficient from the inviscid solutions takes only into account induced drag and these are very consistent results: the maximum error was of less than 1%. Furthermore, by fitting a quadratic polynomial through the inviscid drag polars and by inspection, the Oswald efficiency factor of the wing was indeed equal to 1. This confirmed the elliptical lift distribution and validated the wing aerodynamic performance estimates. In addition, the drag polar from CFD is only shifted upwards by an approximately constant value, which would be the zero-lift drag coefficient, estimated as $C_{D_0} = 0.015$ to provide a sanity check for the following total drag estimation.

7.2 Full UAV Drag Estimation

A full CFD analysis on the UAV was achieved to estimate the drag coefficient at operating conditions for the race [12]. A multitude of issues were encountered, including modifying the CAD model such that it could be read by Star-CCM+ and such that some specific geometrical features could be meshed properly (eg. the trailing edge of the vertical tail, coincident with the symmetry plane). A careful mesh refinement and selection of physics model allowed for results whose accuracy could be questionable [12]. As a result, another method had to be used to confirm and validate the values obtained. A drag build-up method - semi-empirical/analytical - was thus put in place on MATLAB and relied heavily on the method outlined in Raymer [3]. The zero-lift drag coefficient can then be estimated as a sum of contribution from the different components:

$$C_{D_0} = \frac{\Sigma(C_f \, FF \, Q \, S_{wet})_{component}}{S} + C_{D_{leakage}} \tag{7.1}$$

where FF is a form factor that accounts for pressure drag from viscous separation, Q is an interference factor and C_f is the skin friction drag coefficient for a flat plate [3]. The latter can be obtained in the following way:

$$C_{f_{laminar}} = \frac{1.328}{\sqrt{Re}} \quad \text{and} \quad C_{f_{turbulent}} = \frac{0.455}{(\log_{10} Re)^{2.58}} \tag{7.2}$$

The UAV was subsequently broken down in a wing, a horizontal tail, a vertical tail, and a fuselage. A more detailed explanation of the process and the MATLAB code used are presented in Appendix F along with the contribution of the different components to the total drag. The flow was assumed to be fully turbulent over the aircraft for several reasons. Firstly, it would give an overestimate of drag which can be taken as a conservative value. Secondly, the wing would be covered with solar panels whose roughness cannot be estimated.
Indeed, assuming the flow is laminar on some portion of the wing, estimating the transition point proved to be very challenging as it is a very sensitive process. Some analytical and empirical methods were studied, such as Michel's method [20] [21] for low Reynolds numbers (between 10^5 and $40 \cdot 10^6$) and the e^N method on XFoil [8]. These showed very different results and the process to obtain the transition point location from these two methods is outlined in Appendix G. A CFD simulation using the k-omega turbulence model along with a Gamma ReTheta transition model, found to be the most appropriate method on Star-CCM+ [22], was also run on the wing alone. However, it required to input a correlation function via a field function [22] whose accuracy is difficult to estimate. The predicted transition point on the upper surface are shown in the following Table:

Method	e^N, $N_{crit} = 13$ (XFoil)	Michel's method	CFD
$(x/c)_{tr}$	95%	33%	80%

Table 7.1: Transition point location estimation

Furthermore, this is assuming a smooth surface whereas the wing would actually be covered by solar panels whose roughness is unknown and transition would therefore appear earlier. Due to a lack of data available and inconclusive results from the different methods used, it was decided to assume fully turbulent flow over the full aircraft. It would also allow for a better comparison with the results obtained from CFD on the full UAV.
Induced drag was estimated using:

$$C_{D_i} = \frac{C_L^2}{\pi \, \text{Æ} \, e} \tag{7.3}$$

for the wing and where e was equal to 1, as demonstrated in the previous section. For the tail, induced drag was found from:

$$C_{D_{i,tail}} = \frac{S_{tail}}{S} \frac{C_{L_{tail}}^2}{\pi \, \text{Æ}_{tail} \, e_{tail}} \tag{7.4}$$

where $C_{L_{tail}}$ and S_{tail} were given by FATT05 [17], Æ_{tail} was obtained in [13], and e_{tail} was calculated using a formula in [3]. The final drag coefficient values are presented in the following Table:

	Drag build-up			CFD
	C_{D_0}	C_{D_i}	C_D	C_D
Race	0.013	0.0017	0.015	0.020
Application	0.013	0.011	0.024	-

Table 7.2: Drag coefficients estimation

A relatively large difference between the values from CFD and from the drag-build up method can be explained by the inaccuracies of both methods. Indeed, the drag-build up method is more appropriate in an initial sizing phase [3] while the drag coefficients from CFD were found to decrease with increasing mesh density [12] such

that the value presented above from CFD is an overestimate. Furthermore, it was also found to depend on the turbulence model [12]. Therefore, the drag build up method values were used as the level of accuracy on the CFD results could not be estimated in the time frame of this project. Finally, the lift and drag coefficients of the *Solar Flare* are summarized in Table 7.3.

	C_L	C_D	L/D
Race	0.19	0.015	12.7
Application	0.48	0.024	20.0

Table 7.3: Summary of the aerodynamic performance of the UAV at race and application conditions

7.3 Fuselage Nosecone Optimisation

The fuselage nose is meant to house the camera, which is supposed to rotate, for the application. As a result, a practical shape would be a semi-sphere which was used for the first iteration. However, in terms of aerodynamic performance, this corresponds to a bluff body with large drag and high risks of separation. This was observed during the CFD analysis of the full UAV from the first iteration. As a result, a better performing nosecone was generated from the equations in [23] where the aim is to maximize the laminar flow portion over it in order to minimise the drag. The inputs are the base diameter, which is fixed by FATT02 to fit the required systems and payload, and length, which is determined such that the camera can rotate freely inside. This shape, as shown in Figure 7.2, was found to perform better than the previous one as no separation was observed during the CFD analysis of the UAV for the second iteration. The MATLAB code used to generate the shape is presented in Appendix H.

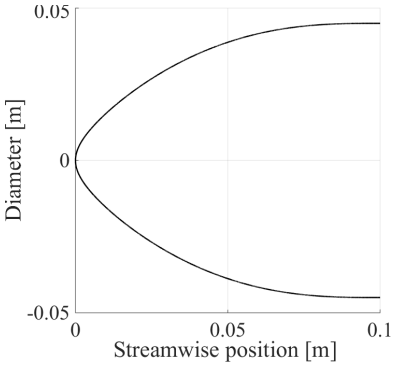


Figure 7.2: Fuselage nose shape

8 Aerodynamic Performance Enhancement Devices

The performance of the UAV having been determined, the next step was to evaluate the possible improvements of using vortex generators and wing tips. Indeed, it was originally thought that vortex generators could delay separation and thus ameliorate the aerodynamics of the wing, especially for application conditions where the operating angles of attack are larger. Furthermore, as the UAV will fly at a larger lift coefficient in the application conditions, the induced drag will be larger. This was shown in Table 7.2 where the induced drag for the application was found to be 540% larger than for the race. Winglets are therefore also considered to improve the performance of the wing over the race and application conditions.

8.1 Vortex Generators

A process of optimisation for vortex generators was implemented during the first iteration. The geometry of these devices was thus optimised on the 1^{st} iteration wing. The first step was to determine the angle of attack at which separation first occurred and its chordwise position. Indeed, vortex generators, when placed slightly before the separation point, create a set of streamwise trailing vortices which bring some of the high-energy flow to the low-energy boundary layer in the near-wall region [24]. Hence the boundary layer becomes turbulent and can stay attached. If the drag penalty due to forcing transition to turbulent flow and to added residual drag from the VGs is lower than the improvement in lift and drag due to the flow staying attached, the overall aerodynamic performance of the wing can be greatly improved, especially at large angles of attack [24]. A VG acts in a similar way to a wing: the more lift it produces, the larger the vortices shed downstream will be. Therefore, they should be oriented to the incoming flow at an angle approaching stall [25]. Indeed, if this angle is too large, more drag will be created due to the added frontal area, but if it is too small, they will not be efficient enough at re-energizing the boundary layer [25].
It was decided to study only vortex generators of the same scale as the boundary layer height, δ, in order to facilitate a possible manufacture and installation of such devices. To estimate the boundary layer thickness, the pressure distribution over the aerofoil, as given by XFoil, was used along with the following expression, for a turbulent boundary layer developing over a flat plate [26]:

$$\delta(x) = \frac{0.385\,x}{Re_x^{1/5}} \tag{8.1}$$

where Re_x is defined as:

$$Re_x = \frac{U_e\,x}{\nu} \tag{8.2}$$

and for which U_e was obtained from the XFoil C_P distribution as:

$$U_e = U_\infty \sqrt{1 - C_P} \tag{8.3}$$

The boundary layer thickness obtained is presented in Figure 8.1.

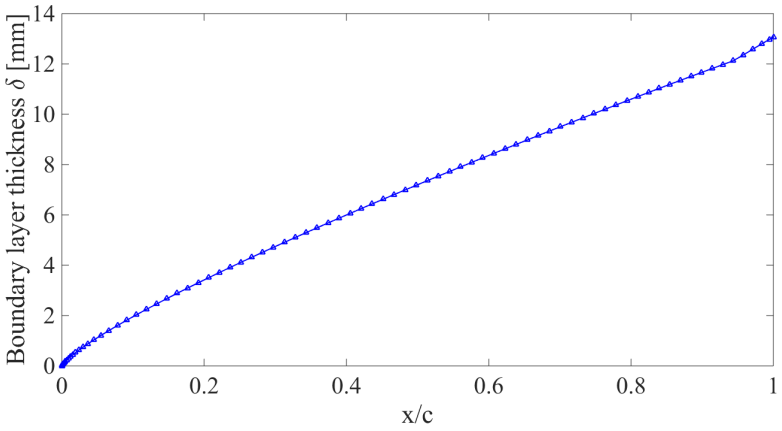


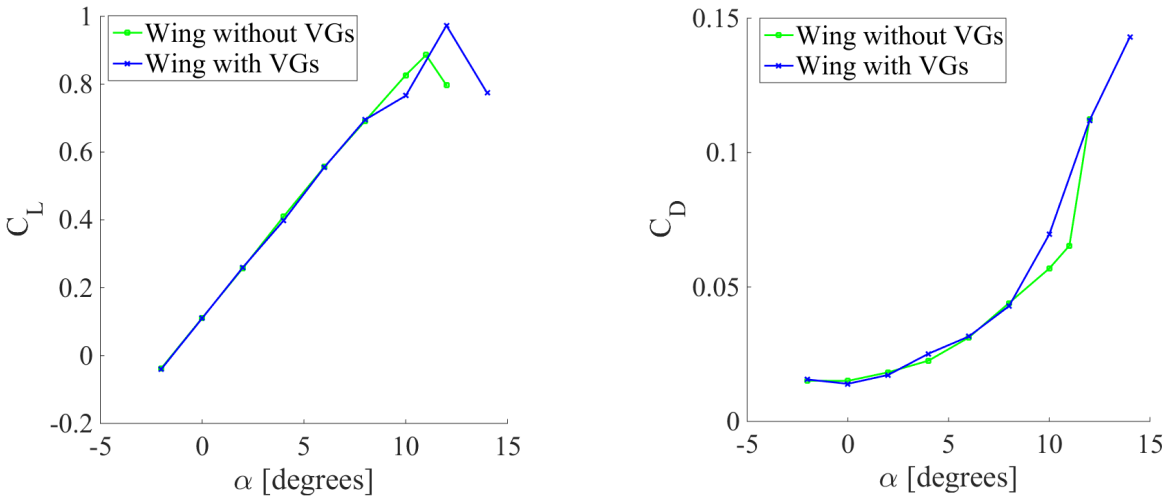
Figure 8.1: Boundary layer thickness

It was then chosen to fix the height of the vortex generators to 8 mm, slightly higher than the boundary layer at the separation point, found to be at $x/c = 40\%$ at 10 ° angle of attack. The determination of the separation point location through flow visualisation using CFD is shown in Appendix I. This is to ensure that the core of the vortex created is at or above the boundary layer in order to efficiently re-energise it [24]. In this report, an optimisation of tapered flat plate vortex generators will be shown. Only the angle to the incoming flow was varied while keeping the other parameters fixed, such as length, thickness, spacing, and chordwise position. Counter-rotating vortex generators were examined first and were later compared to co-rotating ones. This optimisation process was done using Star-CCM+ and the exact same mesh and physics settings for all cases considered allowed for consistent and reliable comparison. This was essential to be able to find the most efficient VG geometry out of all the geometries tested by the different members of FATT03 [11] [13]. A careful mesh refinement around the leading edge, trailing edge and especially around the vortex generators was achieved along with volumetric control around the wing to capture the wake, such that all residuals and calculated parameters converged to a constant value. The Table below summarises the results of this optimisation, where the angle of attack was set at 10° and the velocity at 25 m/s.

	No VGs	Counter-rotating VGs				Co-rotating VGs
Angle to incoming flow	-	10°	15°	20°	25°	20°
C_L	0.700	0.790	0.790	0.790	0.790	0.770
C_D	0.0737	0.0446	0.0447	0.0447	0.0451	0.0505
L/D	9.45	17.68	17.65	17.61	17.52	15.25
Increase in L/D	-	86.1%	85.9%	85.4%	84.4%	60.5%

Table 8.1: Optimisation of the angle to the incoming flow for tapered flat plate vortex generators, based on iteration 1

The counter-rotating vortex generators at the lowest angle of attack tested (10°) were found to most improve the aerodynamic performance. These were then compared to results from other team members and the rectangular flat plate counter-rotating vortex generators at 10° angle of attack were the most efficient, yielding an increase in L/D of 103% [11]. The geometries of these vortex generators are shown in Appendix J.
This geometry was then selected to be placed on the second iteration wing at the separation point, which was much further aft, at $x/c = 61\%$ and for an angle of attack of 12°. This will have a much lower impact on the performance of the wing at large angles. The lift and drag polars obtained with and without vortex generators for the second iteration are shown in Figure 8.2. These were obtained for the race velocity of 16 m/s.
The vortex generators only increased $C_{L_{max}}$ by about 10% leading to a decrease in stall velocity of about 4%. The drag is even slightly higher at larger angles of attack when using vortex generators. The VGs seem to have almost no effect on the aerodynamic performance of the wing in comparison to the results obtained for the first iteration one. This can be due to the separation point being further aft such that the vortex generators' influence is restricted to a smaller portion of the wing. The separation behaviour was also different between the two wings as the Reynolds number decreased. Indeed, a large separated flow region on the upper surface of the wing was observed at 10° angle of attack for the first iteration wing while only a small re-circulation region was seen on the second iteration one. This is shown through flow visualisation in Appendix I using CFD. This can be due to the different aerofoil profile used which leads to a more gentle stall behaviour for the second iteration



(a) Lift polar **(b)** Drag polar

Figure 8.2: Lift and drag polars comparing the performance of the 2^{nd} iteration wing with and without vortex generators

wing. This could have been more accurately observed using a Direct Numerical Simulation (DNS) model which is more accurate but very computationally expensive [27]. Otherwise, an unsteady model and/or a large eddy simulation (LES) could better capture the flow over the wing at large angles of attack [27]. Vortex generators would otherwise be more effective if separation occurred closer to the leading edge as they would influence the flow over the entire wing.
From the results obtained, it was therefore decided to go against using vortex generators, also due to the added complexity in manufacture and installation.

8.2 Wing Tips

Adding a wing tip, if using an appropriate geometry, can considerably reduce induced drag by the interaction of the sidewash created with the natural downwash effect of the wing [28]. However, this effect can be balanced by the increase in wetted area and thus of pressure and skin friction drag. A careful optimisation should therefore be considered. Similarly to the vortex generators optimisation process, evaluating the performance of wing tips was done in parallel with other members of the aerodynamics team [11] [12] [13] using CFD (Star-CCM+) with again the same mesh settings for consistency. The design presented did not go through any optimisation process itself due to the time constraints. The objective of the team was to test a few different wing tip geometries to allow for a feasibility study on whether a wing tip would improve the performance at the application conditions without restrictively decreasing that at the race conditions. The geometries were compared for the first iteration at 8° angle of attack and gave the following results:

Geometry	C_L	C_D	L/D
No wing tip	0.56	0.046	12.0
Hoerner [11]	0.59	0.046	13.7
Upswept [13]	0.62	0.039	16.2
Aft- and upswept	0.60	0.046	12.9

Table 8.2: Comparison of wing tip geometries at 8° angle of attack and at $25\,m/s$, based on iteration 1

All wing tips presented similar drag coefficients but larger lift. As a result, the wing could potentially be set at a lower setting angle to produce the same overall lift, which would reduce the drag. The most efficient wing tip geometry was found to be the upswept one but it was realised later on that this geometry was in fact impossible to manufacture as the thickness of the wing tip was close to zero at the junction with the wing. It was thus decided to use the aft- and upswept wing tip, whose geometry is shown in Figure 8.3.
The flow field around the tip of the wing is clearly not ideal, even if it gives the best performance out of the geometries tested. Indeed, the wing tip should almost eliminate the trailing vortices by canceling their effect with the sidewash produced. However, due to a lack of available time to improve on this design, it was used on the second iteration wing and compared with the wing without wing tips at race and application conditions using the exact same physics and mesh settings on Star-CCM+. Flow visualisation is shown in Appendix K. The results obtained are presented in Table 8.3.

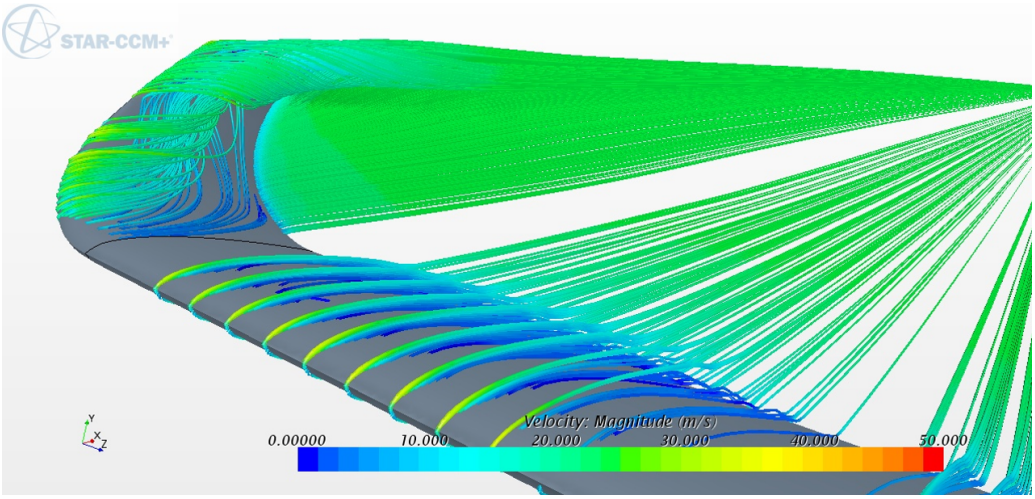


Figure 8.3: Wing tip geometry chosen and flow visualisation using Star-CCM+ on the first iteration wing

Conditions	Geometry	C_L	C_D	L/D
Race	No wing tip	0.19	0.016	11.7
	Wing tip	0.20	0.017	11.7
Application	No wing tip	0.47	0.028	16.4
	Wing tip	0.49	0.031	15.8

Table 8.3: Wing with and without wing tip at application and race conditions, based on iteration 2

The selected wing tip geometry did not improve the performance of the wing, surprisingly decreasing L/D at the application conditions where it was thought that it would have the most beneficial effect. Consequently, no wing tips were added to the final design even though it is believed that, with more time in hand, it would be possible to optimise a geometry that improves the wing's performance as it did for iteration 1.

9 Conclusion

The results obtained from the aerodynamics team were found to be very consistent and were validated using numerous methods, both analytical and computational. CFD analyses were also conducted and allowed to confirm the reliability of inviscid calculations. This was made possible through careful validation of the various methods and softwares used along with a thorough understanding of the assumptions made in all of them. Several optimisation processes, appropriate to the time frame of this project, were then implemented successfully. The aerodynamic performance of the full UAV allows it to be competitive in the Solar Challenge race while staying very efficient for low speed high endurance applications ($L/D = 20$ for the application conditions). Indeed, as confirmed by the other FATTs, its performance arises from the large amount of excess power, in part due to the very low drag force ($2.5\,N$ for the race and $1.7\,N$ for the application).
However, a number of improvements could still be made to obtain more accurate results, especially in terms of drag. More time should be spent on the CFD analyses of the full UAV to judge the effect of mesh dependency and of the physics models used to achieve reliable results. Wind tunnel experiments could then validate the drag values obtained.
Furthermore, the effects of aeroelasticity were not considered. However, very low deflections, less than 1 *cm* [16], were obtained for the second iteration and thus limit the necessity of considering such effects, thus making the calculations presented very accurate.
Finally, to improve the performance of the wing, due to the wide range of Reynolds numbers between the tip and the root, the aerofoil optimisation process could be implemented for different wing sections such as to yield the most appropriate aerofoil at that Reynolds number. A more refined study and optimisation of wing tip geometries would allow for an improved performance over a range of operating conditions or would confirm the results presented in this report. Similarly, to definitely disregard vortex generators, in-depth analyses using both CFD and experimental methods would be required. For instance, using another physics model on Star-CCM+ to model more precisely the unsteady flow around the wing with vortex generators with a more refined mesh would allow for increased accuracy in the results. Furthermore, passive flow control devices were studied and further performance improvements could be achieved with active flow control mechanisms, such as boundary layer suction and blowing [31].

References

[1] R. Hewson, Y. Hwang, R. Palacios, Z. Sharif & M. Santer, *3rd-Year Group Design Project: Project Statement, Solar Challenge 2016*, [document], Department of Aeronautics, Imperial College London, 2016.

[2] RSA, *THE SOLAR CHALLENGE*, [online] Available from: https://www.thersa.org/action-and-research/fellowship-projects/fellowship/the-solar-challenge [Accessed 11/05/2016]

[3] D. Raymer, *Aircraft Design: A Conceptual Approach*, AIAA Education Series. 1989.

[4] *XFLR5: Analysis of foils and wings operating at low Reynolds numbers*, Guidelines for QFLR5 v0.03, October 2009.

[5] M. Hepperle, *Neue Profile für Nurflügelmodelle*, FMT-Kolleg 8, Verlag für Technik und Handwerk, Baden-Baden, Germany, 1988.

[6] M. Selig, J. Guglielmo, A. Broeren & P. Giguere, *Summary of Low-Speed Airfoil Data, Vol. 1*, SoarTech Publications, Virginia Beach, Virginia, 1995.

[7] W. Shyy, Y. Lian, J. Tang, D. Viieru, & H. Liu, *Aerodynamics of Low Reynolds Number Flyers*, Cambridge University Press, 2008.

[8] M. Drela, *XFOIL: An Analysis and Design System for Low Reynolds Number Airfoils*, in "Low Reynolds Number Aerodynamics", Proceedings of the Conference Notre Dame, Indiana, USA, 5–7 June 1989.

[9] R. Evangelista, R. McGhee, & B. Walker, *Correlation of Theory to Wind-Tunnel Data at Reynolds Numbers below 500,000*, in "Low Reynolds Number Aerodynamics", Proceedings of the Conference Notre Dame, Indiana, USA, 5–7 June 1989.

[10] M. Selig, A. Gopalarathnam, P. Giguere, & C. Lyon, *Systematic Airfoil Design Studies at Low Reynolds Numbers*, Presented at the conference on Fixed, Flapping and Rotary Wing Vehicles at Very Low Reynolds Numbers, Notre Dame, IN 46556, June 5–7, 2000.

[11] J. Delpech, *GDP 2016: Solar Challenge, FATT03 Aerodynamic Design*, Department of Aeronautics, Imperial College London, 2016.

[12] J. Ibrahim, *GDP 2016: Solar Challenge, FATT03 Aerodynamic Design*, Department of Aeronautics, Imperial College London, 2016.

[13] S. W. Lee, *GDP 2016: Solar Challenge, FATT03 Aerodynamic Design*, Department of Aeronautics, Imperial College London, 2016.

[14] A. Vivaldi, *GDP 2016: Solar Challenge, FATT01 Configuration Definition*, Department of Aeronautics, Imperial College London, 2016.

[15] G. Konstantellos, *GDP 2016: Solar Challenge, FATT02 Systems and Integration*, Department of Aeronautics, Imperial College London, 2016.

[16] V. Hiscock, *GDP 2016: Solar Challenge, FATT04 Structural Design and Manufacturing*, Department of Aeronautics, Imperial College London, 2016.

[17] S. Y. Ren, *GDP 2016: Solar Challenge, FATT05 Flight Dynamics and Control*, Department of Aeronautics, Imperial College London, 2016.

[18] T. Melin, *User's guide and reference manual for Tornado*, Royal Institute of Technology (KTH), Department of aeronautics, 2000-12.

[19] L. Erickson. *Panel Methods - An Introduction*. NASA Technical Paper 2995. Ames Research Centre, NASA. 1990.

[20] R. Michel, *Calcul Pratique de la Couche Limite Turbulente Compressible*, N.T. O.N.E.R.A. N°49, 1959.

[21] E. Mayda, *Boundary-layer Transition Prediction for Reynolds-averaged Navier-Stokes Methods*, University of California, Davis, 2007.

[22] P. Malan, K. Suluksna, & E. Juntasaro, *Calibrating the $\gamma - Re_\theta$ Transition Model for Commercial CFD*, 47th AIAA Aerospace Sciences Meeting Including The New Horizons Forum and Aerospace Exposition, 2009.

[23] J. Parsons, R. Goodsont, & F. Goldschmied, *Shaping of Axisymmetric Bodies for Minimum Drag in Incompressible Flow*, J. HYDRONAUTICS, VOL.8, NO. 3, 1974.

[24] K. Raykowski, *Optimisation of a Vortex Generator Configuration for a 1/4-Scale Piper Cherokee Wing*, Embry-Riddle Aeronautical University, 1999.

[25] J. Lin, *Review of Research on Low-Profile Vortex Generators to Control Boundary-Layer Separation*, Progress in Aerospace Sciences (ISSN 0376-0421); Volume 38; 389-420, 2002.

[26] H. Schlichting & K. Gersten, *Boundary Layer Theory*, Springer; 8th edition, 2000.

[27] G. Castiglioni, J. Domaradzki, V. Pasquariello, S. Hickel, & M. Grilli, *Numerical Simulations of Separated Flows at Moderate Reynolds Numbers Appropriate for Turbine Blades and Unmanned Aero Vehicles*, Int. J. Heat Fluid Flow, 2004.

[28] M. Maughmer, *The Design of Winglets for Low-Speed Aircraft*, The Pennsylvania State University, 2006.

[29] H. Garner, B. Hewitt, & T. Labrujere, *Comparison of Three Methods for the Evaluation of Subsonic Lifting-Surface Theory*, Reports and Memoranda No. 3597, 1968.

[30] W. Jacobs, *Pressure-Distribution Measurements on Unyawed Swept-Back Wings*, National Advisory Committee for Aeronautics, 1947.

[31] M. S. Genc & Ü. Kaynak, *Control of Laminar Separation Bubble over a NACA2415 Aerofoil at Low Re Transitional Flow Using Blowing/Suction*, 13th International Conference on Aerospace Sciences & Aviation Technology, ASAT- 13, 2009.

Appendix A: Tailless Aircraft Feasibility Study - Aircraft Geometries

The following geometries were used on XFLR5 to evaluate the feasibility of a flying wing configuration design.

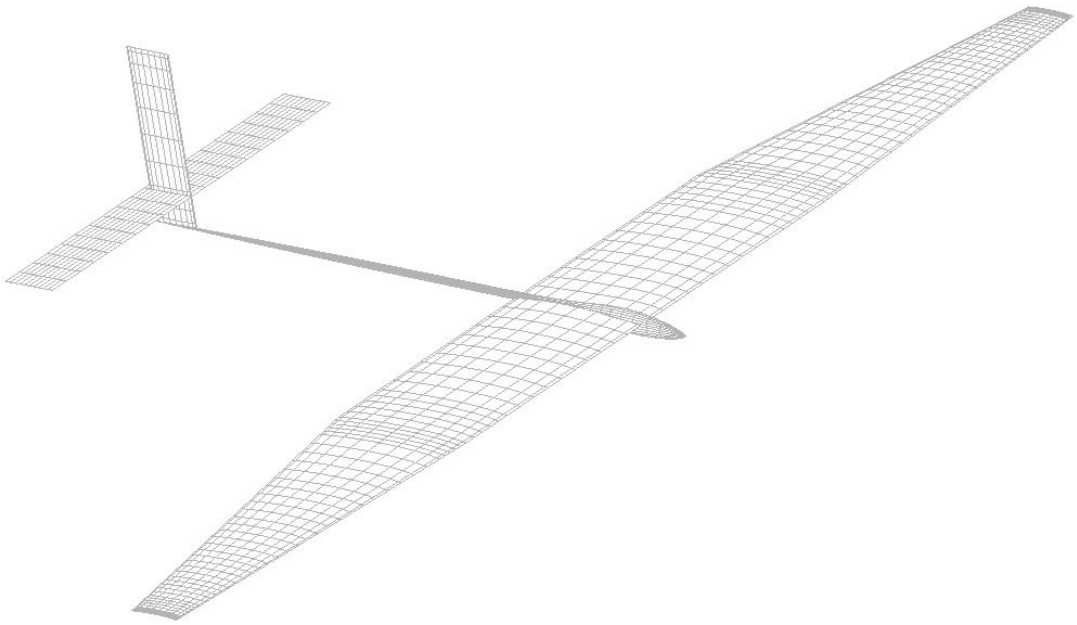


Figure 9.1: Conventional configuration

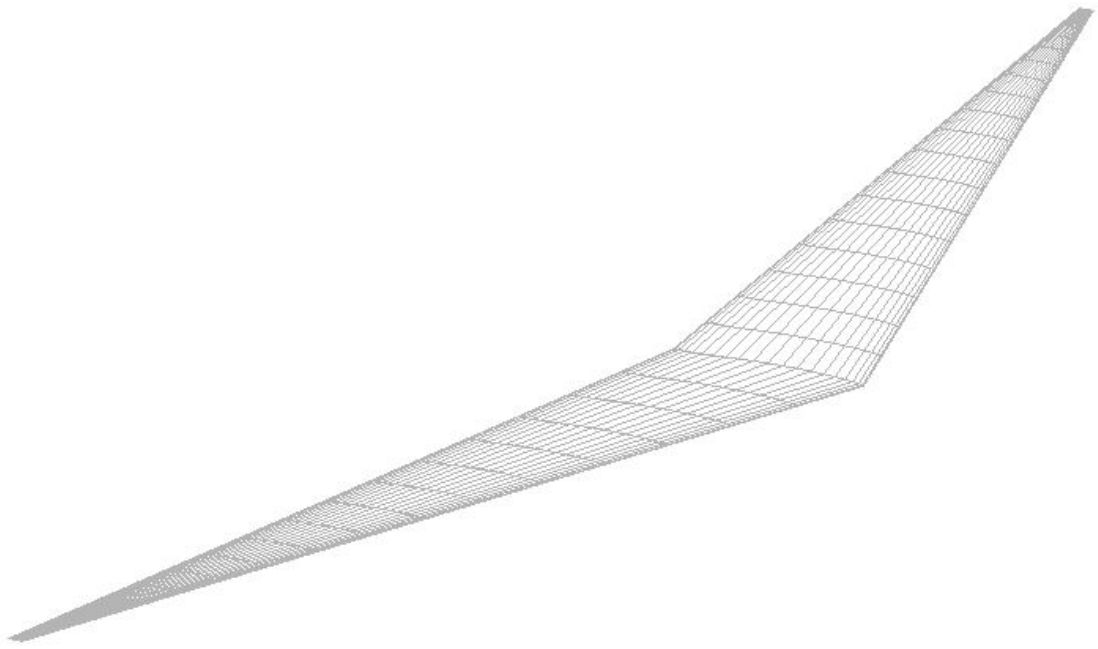


Figure 9.2: Tailless configuration

They were based on initial parameters obtained early on in the design stages. The horizontal and vertical tailplanes were sized according to typical values for tail volume coefficients found in [3]. The wing area and aspect ratio were the same for both configurations, which were then set at the angle of attack required to produce the same overall lift, i.e. to sustain a mass of $3.5\,kg$.

Appendix B: Initial Performance Estimates - Matlab Code for Iteration 2

```
% Written by Lionel Kloetzli - 01/06/2016
%-------------------------------------------------------------------------
% Computes the optimum race performance on a 100m figure-of-eight circuit.
% The inputs are main geometry and performance parameters, some of them
% obtained from other FATTs (FATT01 and FATT02). The circuit is considered
% as 4 segments: acceleration phase, cruise phase, deceleration phase, and
% turning phase. The equations of motions are solved for a constant
% altitude flight. The objective is to find the velocity profile during the
% race that leads to the minimum time required to complete it.
%-------------------------------------------------------------------------
clear
clc
close all

%Input parameters
m=3;
AR=7;
g=9.81;
S=1;
CD0=0.014;
rho=1.225;
v_turn=15;
r_turn=v_turn^2/(g*sqrt(3^2-1)); %load factor of 3

x_acc_max=20;

n=1; %initialise counter

%vary the distance of the acceleration phase
for x_acceleration=1:0.1:x_acc_max
%% initial acceleration
T_acceleration=9; %Maximum thrust that can be produced (given by FATT02)
tspan = [0 5];
x0 = [0 v_turn];
[t,x] = ode45(@(t,x) odefun(t,x,CD0,rho,S,T_acceleration,m,AR,g),tspan,x0);
%solve the equation of motion in "odefun" for acceleration conditions

t=t';
x=x';

%find the index corresponding to the time required to complete a distance
%x_acceleration during this phase
i_acceleration=find(abs(x(1,:)-x_acceleration)==min(abs(x(1,:)-...
    x_acceleration)));
t_acceleration=t(i_acceleration);

%Find the time and velocity history corresponding to this acceleration
%phase
v_acc{n}=x(2,1:i_acceleration);
t_acc{n}=t(1:i_acceleration);

%% cruise
V_cruise(n)=x(2,i_acceleration);
%Cruise velocity is the velocity at the end of the acceleration phase
clear t x tspan x0

%% decceleration
T_decceleration=0; %0 thrust during the deceleration phase
tspan = [0 10];
x0 = [0 V_cruise(n)];
[t,x] = ode45(@(t,x) odefun(t,x,CD0,rho,S,T_decceleration,m,AR,g),tspan,x0);
%solve the equations of motion for the deceleration phase through "odefun"
t=t';
x=x';

%find the index corresponding to the distance required to reach a velocity
%v_turn at the end of the deceleration phase
i_decceleration=find(abs(x(2,:)-v_turn)==min(abs(x(2,:)-v_turn)));

%find the time required to decelerate and the corresponding distance
t_decceleration=t(i_decceleration);
```

```
x_decceleration=x(1,i_decceleration);

%find the cruise distance from the acceleration and deceleration distances
%and the geometry of the circuit from turning conditions
x_cruise(n)=sqrt(100^2+4*r_turn^2)-(x_acceleration+x_decceleration);
if x_cruise(n)<0
    %if the cruise distance is negative, the deceleration phase is too long
    %such that there is no more solution for this race profile
    break
end

%otherwise, the time from the cruise segment is obtained from the cruise
%distance and velocity
t_cruise(n)=x_cruise(n)./V_cruise(n);

%Store the velocity and time history of the deceleration phase
v_dec{n}=x(2,1:i_decceleration);
t_dec{n}=t(1:i_decceleration);

%% turn
t_turn=pi*r_turn/v_turn;
%find the time required to achieve the turn, assuming constant velocity

%% total

%The total time is twice the sum of the acceleration, cruise, deceleration
%and turning phases to complete the full circuit
t_total(n)=2*(t_acceleration+t_cruise(n)+t_decceleration+t_turn);

x_acc(n)=x_acceleration;
n=n+1;
end

n=n-1;
%generate plot of the velocity profile for the optimum race time
figure
set(gcf,'color','w')
set(0,'defaultaxesfontname','Times New Roman')
hold all
plot(t_acc{n},v_acc{n},'k','linewidth',3)
plot([t_acc{n}(end),t_acc{n}(end)+t_cruise(n)],[V_cruise(n),V_cruise(n)],...
    ':k','linewidth',3)
plot(t_acc{n}(end)+t_cruise(n)+t_dec{n},v_dec{n},'--k','linewidth',3)
plot([t_acc{n}(end)+t_cruise(n)+t_dec{n}(end),t_acc{n}(end)+t_cruise(n)+...
    t_dec{n}(end)+t_turn],[v_turn,v_turn],'-.k','linewidth',3)
xlabel('Time [s]')
ylabel('Velocity [m/s]')
set(gca,'fontsize',20)
l=legend('Acceleration','Cruise','Deceleration','Turn');
set(l,'fontsize',20)
ylim([14.5 18])

%compute the average velocity for this race profile
average_velocity=(trapz(t_acc{n},v_acc{n})+trapz([t_acc{n}(end),...
    t_acc{n}(end)+t_cruise(n)],[V_cruise(n),V_cruise(n)])+...
    trapz(t_acc{n}(end)+t_cruise(n)+t_dec{n},v_dec{n})+...
    trapz([t_acc{n}(end)+t_cruise(n)+t_dec{n}(end),t_acc{n}(end)+...
    t_cruise(n)+t_dec{n}(end)+t_turn],[v_turn,v_turn]))/(0.5*t_total(n))
```

```
function dxdt = odefun(t,x,CD0,rho,S,T,m,AR,g)
%This function sets up the equations of motion to be solved in the main
%code

dxdt=zeros(2,1);
CL=2*m*g./(rho*x(2)^2*S);
CDi=CL^2/(pi*AR);
D=0.5*rho*x(2)^2*S*(CD0+CDi); %Generate drag force

dxdt(1) = x(2);
dxdt(2) = (T-D)/m;
end
```

Appendix C: Aerofoil Optimisation - Matlab Code

```
% Written by Lionel Kloetzli and Jules Delpech - May 2016
%--------------------------------------------------------------------------
% This code compares a set of pre-selected aerofoils using XFoil and finds
% the most appropriate one from a Merit function that can be tailored to
% the need of the user
%--------------------------------------------------------------------------

clear
clc
close all

%Initial conditions (Reynolds number and Mach number)
Re=4.1e5;
Mach=0.05;

%% Non-symmetric NACA Airfoils (4-series)

camber=[1:2];
camberposition=[2:8];
thickness=[10:12];
NACA={};

for i=1:length(camber)
    for j=1:length(camberposition)
        for k=1:length(thickness)
            NACA=[NACA,char([num2str(camber(i)),num2str(camberposition(j)),...
            num2str(thickness(k))])];
        end
    end
end
for i=1:length(NACA)
    Airfoil_name(i)={['NACA',num2str(NACA{i})]};
end

%% Symmetric NACA airfoils (4-series)

camber=[0];
camberposition=[0];
thickness=[10:12];
for i=1:length(camber)
    for j=1:length(camberposition)
        for k=1:length(thickness)
            if thickness(k)<10
                NACA=[NACA,char([num2str(camber(i)),num2str(camberposition(j)),'0',...
                num2str(thickness(k))])];
            else
                NACA=[NACA,char([num2str(camber(i)),num2str(camberposition(j)),...
                num2str(thickness(k))])];
            end
        end
    end
end
for i=1:length(NACA)
    Airfoil_name(i)={['NACA',num2str(NACA{i})]};
end

%% NACA 5-series Airfoils

%Input range of camber, camber position and thickness
camber=[1:2];
camberposition=[2:5];
thickness=[10:12];

for i=1:length(camber)
    for j=1:length(camberposition)
        for k=1:length(thickness)
            NACA=[NACA,char([num2str(camber(i)),num2str(camberposition(j)),num2str(0),...
            num2str(thickness(k))])];
        end
    end
end
for i=1:length(NACA)
```

```
    Airfoil_name(i)={['NACA',num2str(NACA{i})]};
end

%generate angle of attack range
alpha=[-2:0.5:13]; %in degrees

index0=find(alpha==0);
index2=find(alpha==2);
index4=find(alpha==4);
index5=find(alpha==5);
index6=find(alpha==6);
index8=find(alpha==8);

for i=1:length(Airfoil_name)

    %Call XFoil to generate lift, drag and pitching moment values for the
    %range of angles of attack selected, Reynolds number, and Mach number.
    %N_crit is set at 13 and 500 iterations are allowed to maximise the
    %chances of convergence.
    [pol{i},foil{i}] = xfoil(char(Airfoil_name(i)),alpha,Re,Mach,'panels n 500',...
    'oper iter 500','oper/vpar n 13');


    P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');
end
clear P

%-------------------------------------------------------------------------

%% All non-NACA aerofoils

GOE448=importdata('goe448.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'GOE448'};
    [pol{i},foil{i}] = xfoil(GOE448,alpha,Re,Mach,'panels n 300','oper iter 500');


    P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

S1223=importdata('S1223.dat');
```

```
149
150
151 i=length(Airfoil_name)+1;
152     Airfoil_name(i)={'S1223'};
153     [pol{i},foil{i}] = xfoil(S1223,alpha,Re,Mach,'panels n 300','oper iter 500');
154
155
156         P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
157     Cl_mean(i)=P(1)*2+P(2);
158
159     Clmax(i)=max(pol{i}.CL);
160
161     P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
162     a0(i)=180/pi*P(1);
163
164     indexstall=find(pol{i}.CL==Clmax(i),1);
165     Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');
166
167     alphastall(i)=pol{i}.alpha(indexstall);
168
169     Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));
170
171     Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
172     pol{i}.CD(index0:index4)');
173
174 %-------------------------------------------------------------------------
175
176 SD7037=importdata('SD7037.dat');
177
178
179 i=length(Airfoil_name)+1;
180     Airfoil_name(i)={'SD7037'};
181     [pol{i},foil{i}] = xfoil(SD7037,alpha,Re,Mach,'panels n 300','oper iter 500');
182
183
184        P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
185     Cl_mean(i)=P(1)*2+P(2);
186
187     Clmax(i)=max(pol{i}.CL);
188
189     P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
190     a0(i)=180/pi*P(1);
191
192     indexstall=find(pol{i}.CL==Clmax(i),1);
193     Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');
194
195     alphastall(i)=pol{i}.alpha(indexstall);
196
197     Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));
198
199     Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
200     pol{i}.CD(index0:index4)');
201
202 %-------------------------------------------------------------------------
203
204 S7055=importdata('S7055.dat');
205
206
207 i=length(Airfoil_name)+1;
208     Airfoil_name(i)={'S7055'};
209     [pol{i},foil{i}] = xfoil(S7055,alpha,Re,Mach,'panels n 300','oper iter 500');
210
211
212         P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
213     Cl_mean(i)=P(1)*2+P(2);
214
215     Clmax(i)=max(pol{i}.CL);
216
217     P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index68))',1);
218     a0(i)=180/pi*P(1);
219
220     indexstall=find(pol{i}.CL==Clmax(i),1);
221     Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');
222
223     alphastall(i)=pol{i}.alpha(indexstall);
224
```

```
    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

S3021=importdata('S3021.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'S3021'};
    [pol{i},foil{i}] = xfoil(S3021,alpha,Re,Mach,'panels n 300','oper iter 500');


       P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

K3311=importdata('K3311.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'K3311'};
    [pol{i},foil{i}] = xfoil(K3311,alpha,Re,Mach,'panels n 300','oper iter 500');


       P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

FX63137=importdata('FX63137.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'FX63137'};
    [pol{i},foil{i}] = xfoil(FX63137,alpha,Re,Mach,'panels n 300','oper iter 500');


       P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);
```

```
    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

E423=importdata('E423.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'E423'};
    [pol{i},foil{i}] = xfoil(E423,alpha,Re,Mach,'panels n 300','oper iter 500');


    P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

E214=importdata('E214.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'E214'};
    [pol{i},foil{i}] = xfoil(E214,alpha,Re,Mach,'panels n 300','oper iter 500');


      P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

E374=importdata('E374.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'E374'};
```

```
    [pol{i},foil{i}] = xfoil(E374,alpha,Re,Mach,'panels n 300','oper iter 500');


      P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

SD6060=importdata('SD6060.dat');


i=length(Airfoil_name)+1;
    Airfoil_name(i)={'SD6060'};
    [pol{i},foil{i}] = xfoil(SD6060,alpha,Re,Mach,'panels n 300','oper iter 500');


      P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

%-------------------------------------------------------------------------

HQ2010=importdata('HQ20_10.dat');

i=length(Airfoil_name)+1;
    Airfoil_name(i)={'HQ2.0/10'};
    [pol{i},foil{i}] = xfoil(HQ2010,alpha,Re,Mach,'panels n 300','oper iter 500');


      P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
    Cl_mean(i)=P(1)*2+P(2);

    Clmax(i)=max(pol{i}.CL);

    P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
    a0(i)=180/pi*P(1);

    indexstall=find(pol{i}.CL==Clmax(i),1);
    Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');

    alphastall(i)=pol{i}.alpha(indexstall);

    Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));

    Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
    pol{i}.CD(index0:index4)');

```

```
453 %-------------------------------------------------------------------------
454
455 HQ1010=importdata('HQ10_10.dat');
456
457 i=length(Airfoil_name)+1;
458     Airfoil_name(i)={'HQ1.0/10'};
459     [pol{i},foil{i}] = xfoil(HQ1010,alpha,Re,Mach,'panels n 300','oper iter 500');
460
461
462       P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
463     Cl_mean(i)=P(1)*2+P(2);
464
465     Clmax(i)=max(pol{i}.CL);
466
467     P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
468     a0(i)=180/pi*P(1);
469
470     indexstall=find(pol{i}.CL==Clmax(i),1);
471     Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');
472
473     alphastall(i)=pol{i}.alpha(indexstall);
474
475     Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));
476
477     Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
478     pol{i}.CD(index0:index4)');
479
480 %-------------------------------------------------------------------------
481
482 E226=importdata('E226.dat');
483
484
485 i=length(Airfoil_name)+1;
486     Airfoil_name(i)={'E226'};
487     [pol{i},foil{i}] = xfoil(E226,alpha,Re,Mach,'panels n 300','oper iter 500');
488
489
490       P=polyfit(pol{i}.alpha(index0:index2)',(pol{i}.CL(index0:index2))',1);
491     Cl_mean(i)=P(1)*2+P(2);
492
493     Clmax(i)=max(pol{i}.CL);
494
495     P=polyfit(alpha(index0:index6),(pol{i}.CL(index0:index6))',1);
496     a0(i)=180/pi*P(1);
497
498     indexstall=find(pol{i}.CL==Clmax(i),1);
499     Cd_int(i)=trapz(pol{i}.alpha(index0:index8),pol{i}.CD(index0:index8)');
500
501     alphastall(i)=pol{i}.alpha(indexstall);
502
503     Cm(i)=trapz(pol{i}.alpha(index0:index4),abs(pol{i}.Cm(index0:index4)'));
504
505     Cl_Cd_int(i)=trapz(pol{i}.alpha(index0:index4),pol{i}.CL(index0:index4)'./...
506     pol{i}.CD(index0:index4)');
507
508 %-------------------------------------------------------------------------
509
510 %% Normalize values and generate the merit function to maximise
511
512 %average value of lift coefficient at low angles of attack
513 Cl_mean_normalized=Cl_mean./max(Cl_mean);
514
515 %lift curve slope
516 a0_normalized=a0./max(a0);
517
518 %integral of drag coefficient
519 Cd_int_normalized=Cd_int./max(Cd_int);
520
521 %maximum lift coefficient
522 Clmax_normalized=Clmax./max(Clmax);
523
524 %stall angle
525 alphastall_normalized=alphastall./max(alphastall);
526
527 %integral of pitching moment coefficient
528 Cm_normalized=Cm./max(Cm);
```

```
529
530 %integral of lift to drag ratio
531 Cl_Cd_normalized=Cl_Cd_int./max(Cl_Cd_int);
532
533 %generate merit function
534 Meritfunction=(Cm_normalized.*a0_normalized.*Cl_Cd_normalized)./...
535 (Cd_int_normalized);
536
537 %Find optimum aerofoil
538 INDEX=find(Meritfunction==max(Meritfunction),1);
539
540 disp(['Best airfoil: ',Airfoil_name{INDEX}])
541
542 iclmax=find(Clmax_normalized==1,1);
543 iclmean=find(Cl_mean_normalized==1,1);
544 ia0=find(a0_normalized==1,1);
545 ialphastall=find(alphastall_normalized==1,1);
546
547 %% Generate plots
548 figure
549 set(gcf,'color','w')
550 set(0,'defaultaxesfontname','Times New Roman')
551 hold all
552 plot(pol{INDEX}.alpha,pol{INDEX}.CD,'-xr','linewidth',2.5,'markersize',10)
553 plot(pol{iclmax}.alpha,pol{iclmax}.CD,'^b','linewidth',1.5)
554 plot(pol{iclmean}.alpha,pol{iclmean}.CD,'xg','linewidth',1.5)
555 plot(pol{ia0}.alpha,pol{ia0}.CD,'om','linewidth',1.5)
556
557 for i=1:length(Meritfunction)
558     hold on
559     plot(pol{i}.alpha,pol{i}.CD,':k','linewidth',1.2)
560 end
561
562 plot(pol{iclmax}.alpha,pol{iclmax}.CD,'b','linewidth',1.5)
563 plot(pol{iclmean}.alpha,pol{iclmean}.CD,'g','linewidth',1.5)
564 plot(pol{ia0}.alpha,pol{ia0}.CD,'m','linewidth',1.5)
565 plot(pol{INDEX}.alpha,pol{INDEX}.CD,'-r','linewidth',2.5)
566 l=legend(['Selected Airfoil: ',num2str(Airfoil_name{INDEX})],...
567 'Other airfoils tested');
568 set(l,'interpreter','LaTex','fontsize',30)
569 xlabel('$\alpha$ [degrees]','interpreter','LaTex')
570 ylabel('$C_d$','interpreter','LaTex')
571 set(gca,'fontsize',30)
572 title('Drag polar for tested airfoils','fontsize',30)
573 axis square
574
575 figure
576 hold all
577 set(gcf,'color','w')
578 set(0,'defaultaxesfontname','Times New Roman')
579 plot(pol{INDEX}.alpha,pol{INDEX}.CL,'-xr','linewidth',2.5,'markersize',10)
580 plot(pol{iclmax}.alpha,pol{iclmax}.CL,'^b','linewidth',1.5)
581 plot(pol{iclmean}.alpha,pol{iclmean}.CL,'xg','linewidth',1.5)
582 plot(pol{ia0}.alpha,pol{ia0}.CL,'om','linewidth',1.5)
583 plot(pol{ialphastall}.alpha,pol{ialphastall}.CL,'c')
584
585 for i=1:length(Meritfunction)
586     hold on
587     plot(pol{i}.alpha,pol{i}.CL,':k','linewidth',1.2)
588 end
589
590 plot(pol{iclmax}.alpha,pol{iclmax}.CL,'b','linewidth',1.5)
591 plot(pol{iclmean}.alpha,pol{iclmean}.CL,'g','linewidth',1.5)
592 plot(pol{ia0}.alpha,pol{ia0}.CL,'m','linewidth',1.5)
593 plot(pol{INDEX}.alpha,pol{INDEX}.CL,'-r','linewidth',2.5)
594 l=legend(['Selected Airfoil: ',num2str(Airfoil_name{INDEX})],...
595 'Other airfoils tested');
596 set(l,'interpreter','latex','fontsize',30)
597 xlabel('$\alpha$ [degrees]','interpreter','LaTex')
598
599 ylabel('$C_l$','interpreter','latex')
600 set(gca,'fontsize',30)
601 title('Lift polar for tested airfoils','fontsize',30)
602 axis square
603
604 figure
```

```
hold all
set(gcf,'color','w')
set(0,'defaultaxesfontname','Times New Roman')
plot(pol{INDEX}.alpha,pol{INDEX}.Cm,'-xr','linewidth',2.5,'markersize',10)
plot(pol{iclmax}.alpha,pol{iclmax}.Cm,'^b','linewidth',1.5)
plot(pol{iclmean}.alpha,pol{iclmean}.Cm,'xg','linewidth',1.5)
plot(pol{ia0}.alpha,pol{ia0}.Cm,'om','linewidth',1.5)
plot(pol{ialphastall}.alpha,pol{ialphastall}.CL,'c')

for i=1:length(Meritfunction)
    hold on
    plot(pol{i}.alpha,pol{i}.Cm,':k','linewidth',1.2)
end

plot(pol{iclmax}.alpha,pol{iclmax}.Cm,'b','linewidth',1.5)
plot(pol{iclmean}.alpha,pol{iclmean}.Cm,'g','linewidth',1.5)
plot(pol{ia0}.alpha,pol{ia0}.Cm,'m','linewidth',1.5)
plot(pol{INDEX}.alpha,pol{INDEX}.Cm,'-r','linewidth',2.5)
l=legend(['Selected Airfoil: ',num2str(Airfoil_name{INDEX})],...
'Other airfoils tested');
set(l,'interpreter','latex','fontsize',30)
xlabel('$\alpha$ [degrees]','interpreter','LaTex')

ylabel('$C_m$','interpreter','latex')
set(gca,'fontsize',30)
title('Pitching moment coefficient variation with angle of attack for tested airfoils','fontsize',30)
axis square

%% Values for selected aerofoil

lift_curve_slope=a0(INDEX)
CLMAX=Clmax(INDEX)
ASTALL=alphastall(INDEX)
CL0DEG=pol{INDEX}.CL(find(pol{INDEX}.alpha==0))
```

Appendix D: Wing Geometry Design Process - Matlab Code

```
% Written by Lionel Kloetzli, Jules Delpech and Sang Woo Lee - May 2016
%-------------------------------------------------------------------------
% Optimisation of wing geometry for minimum twist required to achieve an
% elliptical loading. This process is based on the Lifting Line Theory.
%-------------------------------------------------------------------------

clc
clear
close all

%% PART 1 : INITIAL GEOMETRY CHOICE
Uinf =16;                                %in m/s
S_ref_wing = 1;                          %in m^2
Weight = 3;                              %In kg
AR = 7;                                  %Aspect ratio
CL_required = Weight * 9.81 * 2 /(Uinf)^2 / S_ref_wing /1.225;
b  = (AR*S_ref_wing).^0.5;               %Total Span, b
s=b/2;                                   %Semi Span, s
c_mean = S_ref_wing./b;                  %mean chord ratio, c_mean

CL= CL_required;

%Airfoil properties
ao= 6.11;                %FINAL ITERATION: NACA 1810
alpha0 = -1.10 * pi/180;         %in Radians
Cl_0 = 0.1625;
t_c = 0.10;                      %thickness to chord ratio
Re = Uinf * c_mean / 1.5E-05;    %Reynolds number based on mean chord
%% PART 2 - Optimisation of planform geometry
%1. Find the best chord distribution (rectangular + tapered) for each tip chord length
%2. Compare the total (integrated) twist for the best chord distribution
%for each tip chord length and find the one with least twist.
%--> This finds the relative twist (0 angle of attack at the root)

c_tip = [0.05:0.01:0.25];
for k=1:length(c_tip)

weight=Weight*9.81;
p=0:0.1:1;
for i=1:length(p)
    g(i)=p(i).*s;
    %g is the spanwise location of the boundary between the constant chord
    %section and the linearly tapered one.
    c_root(k,i)=((S_ref_wing*0.5)-c_tip(k).*(s-g(i)).*0.5).*...
        (g(i)+(s-g(i)).*0.5)^-1;
    y1{i}=[0:0.005:g(i)];
    y2{i}=[g(i):0.005:s];
    c1{k,i}=(y1{i}+1)./(y1{i}+1).*c_root(k,i);

if g(i)==s
    c2{k,i}=c1{k,i}(1);
else
    c2{k,i}=((c_tip(k)-c_root(k,i))./(s-g(i))).*y2{i}+c_root(k,i)-g(i).*...
    (c_tip(k)-c_root(k,i))./(s-g(i));
end

%compute the twist from LLT for an elliptical loading
twist1{k,i}=(CL/(pi*AR)).*(((8.*s)./ao).*(sqrt((1-(y1{i}./s).^2))./(c1{k,i}))+1);
twist2{k,i}=(CL/(pi*AR)).*(((8.*s)./ao).*(sqrt((1-(y2{i}./s).^2))./(c2{k,i}))+1);
twist1{k,i}=180./pi.*(twist1{k,i});
twist2{k,i}=180./pi.*(twist2{k,i});

%integrate the twist distributions
mean(k,i)=(sum(abs(twist1{k,i}-twist1{k,i}(1)))+sum(abs(twist2{k,i}-twist1{k,i}(1)))).*/s;

end

%plot the relative twist and chord distributions for the selected
%geometries
figure (1)
[i1,i2]=find(mean==min(min(mean)));
plot(y1{i2},twist1{k,i2}-twist1{k,i2}(1))
```

```
hold on
plot(y2{i2},twist2{k,i2}-twist1{k,i2}(1))

figure (2)
plot(y1{i2},c1{k,i2})
hold on
plot(y2{i2},c2{k,i2})

end

%find the indeces of the minimum integrated relative twist
[i1,i2]=find(mean==min(min(mean)));

figure (1)
plot(y1{i2},twist1{i1,i2}-twist1{i1,i2}(1),'k','linewidth',2)
hold on
plot(y2{i2},twist2{i1,i2}-twist1{i1,i2}(1),'k','linewidth',2)

figure (2)
plot(y1{i2},c1{i1,i2},'k','linewidth',2)
hold on
plot(y2{i2},c2{i1,i2},'k','linewidth',2)

%plot the relative twist and chord ditribution of the optimum wing geometry
figure
set(gcf,'color','w')
plot(y1{i2},twist1{i1,i2}-twist1{i1,i2}(1),'k','linewidth',2)
hold on
plot(y2{i2},twist2{i1,i2}-twist1{i1,i2}(1),'k','linewidth',2)

Relative_twist=[twist1{i1,i2}-twist1{i1,i2}(1),twist2{i1,i2}-twist1{i1,i2}(1)];
y_index=[y1{i2},y2{i2}];

figure
set(gcf,'color','w')
plot(y1{i2},c1{i1,i2},'k','linewidth',2)
hold on
plot(y2{i2},c2{i1,i2},'k','linewidth',2)

%% PART 3 : LIFTING LINE THEORY - COLLOCATION METHOD
%Find lift coefficients for various angle of attack for the
%optimum wing geometry

%Input the number of panels required to create a file storing the necessary
%wing geometry for Tornado
N= input('number of panels in even number? - twice the number of partitions used in tornado:');

%AoA is a range of angles of attack
AoA=[-2:0.05:15]*pi/180;

for a=1:length(AoA)     %Vary AOA and compute CL using LLT.

    %initialise the collocation method
    for n=1:N
        theta(n)=n*pi/(N+1);
        y(n)=s.*cos(theta(n));
            if abs(y(n))<g(i2)
                c(n)=c_root(i1,i2);
                %alpha is the relative twist + setting angle for various
                %AOA - zero lift angle
                alpha(n)=(CL/(pi*AR)).*(((8.*s)./ao).*...
                    (sqrt((1-(y(n)./s).^2))./(c(n)))+1)+AoA(a)-...
                    (CL/(pi*AR))*(1+(8*s)/(c_root(i1,i2)*ao))-alpha0 ;
            else
                c(n)=((c_tip(i1)-c_root(i1,i2))./(s-g(i2))).*abs(y(n))+...
                    c_root(i1,i2)-g(i2).*(c_tip(i1)-c_root(i1,i2))./(s-g(i2));
                alpha(n)=(CL/(pi*AR)).*(((8.*s)./ao).*...
                    (sqrt((1-(y(n)./s).^2))./(c(n)))+1)+AoA(a)-...
                    (CL/(pi*AR))*(1+(8*s)/(c_root(i1,i2)*ao))-alpha0 ;
            end

        mu(n)=ao.*c(n)./(8.*s);
        rhs(n,1)=mu(n).*alpha(n).*sin(theta(n));

            for m=1:N
                lhs(n,m)=sin(m.*theta(n)).*(m.*mu(n)+sin(theta(n)));
```

```
            end

        A{a}=lhs\rhs; %find the coefficients A1, A2, etc...
    end
end

for a=1:length(AoA)
    sum{a}=zeros(1,N);
        for m=1:N
            for n=1:N
                sum{a}(1,m)=(A{a}(n,1))*sin(n*theta(m))+sum{a}(1,m);
            end
            Cl(a,m)=8.*s*sum{a}(1,m)./c(m);      %sectional lift coefficient
            gamma(a,m)=4*s*Uinf*sum{a}(1,m);     %sectional gamma
        end
end

y= [b/2,y,-b/2];
zero_vector= [zeros(length(AoA),1)];
Cl = [zero_vector,Cl,zero_vector];
gamma = [zero_vector,gamma,zero_vector];
%Plot Cl and gamma distribution for every AoA setting
figure
surf(y,AoA*180/pi,Cl)
set(gcf,'color','w')

figure
surf(y,AoA*180/pi,gamma)
set(gcf,'color','w')

%% FIND WING LIFT COEFFICIENT CL USING LLT
%FROM THAT FIND CDi WING INDUCED DRAG

CDi=zeros(1,length(AoA));
for i=1:length(AoA)
    CL3D(i)=A{i}(1,1).*pi.*AR;
    L(i)=0.5*1.225*S_ref_wing*Uinf^2*CL3D(i);

    for j=1:2:N
        CDi(i)=(j.*(A{1,i}(j,1)).^2)+CDi(i);
    end
    CDi(i)=CDi(i)*pi*AR;
end



%% For Tornado

%This is only to compute the twist and chord distribution for the whole span
%This is because LLT doesn't compute them at the tips.
for a=1:length(AoA)

for n=1:N+1
theta(n)=(n-1)*pi/(N);
y_tornado(n)=s.*cos(theta(n));
if abs(y_tornado(n))<g(i2)
    c(n)=c_root(i1,i2);
    %alpha is the relative twist + setting angle for various AOA (from 0 to
    %15 degrees) - zero lift angle
    alpha_tornado(n)=(CL/(pi*AR)).*(((8.*s)./ao).*...
        (sqrt((1-(y_tornado(n)./s).^2))./(c(n)))+1)+AoA(a)-...
        (CL/(pi*AR))*(1+(8*s)/(c_root(i1,i2)*ao))-alpha0 ;

else
    c(n)=((c_tip(i1)-c_root(i1,i2))./(s-g(i2))).*...
        abs(y_tornado(n))+c_root(i1,i2)-g(i2).*(c_tip(i1)-c_root(i1,i2))./(s-g(i2));
    alpha_tornado(n)=(CL/(pi*AR)).*(((8.*s)./ao).*...
        (sqrt((1-(y_tornado(n)./s).^2))./(c(n)))+1)+AoA(a)-...
        (CL/(pi*AR))*(1+(8*s)/(c_root(i1,i2)*ao))-alpha0 ;

end
end
end

rel_twist_deg_tornado = (alpha_tornado - AoA(end)+alpha0)*180/pi;
```

```
number = N/2+1;
y_semi = fliplr(y_tornado(1:number));
c_semi = fliplr(c(1:number));
rel_twist_deg_semi = fliplr(rel_twist_deg_tornado(1:number));

for i = 2:length(y_semi);
y_interval(i) = y_semi(i) - y_semi(i-1);
end

for i = 2:length(c_semi);
taper_interval(i) = c_semi(i) / c_semi(i-1);
end

span_partition_tornado = y_interval(2:end);
taper_partition_tornado = taper_interval(2:end);
twist_outboard_tornado_1 = rel_twist_deg_semi(1:end-1)*pi/180;
twist_outboard_tornado_2 = rel_twist_deg_semi(2:end)*pi/180;

%Store the data into one variable, as computed in the format required by
%Tornado

geo.flapped=zeros(1,N/2);
geo.nwing=1;
geo.nelem=N/2;
geo.CG=[0,0,0];
geo.ref_point=[0,0,0];
geo.symetric=1;
geo.startx=0;
geo.starty=0;
geo.startz=0;
geo.c=max(c);

for i=1:geo.nelem
   for j=1:2
       geo.foil{1,i,j}='0010';
   end
end

geo.nx=(5).*ones(1,geo.nelem);

for i=1:geo.nelem
    geo.TW{1,i,1}=twist_outboard_tornado_1(i);
end

for i=1:geo.nelem
       geo.TW{1,i,2}=twist_outboard_tornado_2(i);
end

geo.TW = cell2mat(geo.TW);
geo.dihed=zeros(1,geo.nelem);
geo.ny=(round(100/geo.nelem)).*ones(1,geo.nelem);
geo.b=span_partition_tornado;
geo.T=taper_partition_tornado;
geo.SW=zeros(1,geo.nelem);
geo.meshtype=ones(1,geo.nelem);
geo.fc=zeros(1,geo.nelem);
geo.fnx=zeros(1,geo.nelem);
geo.fsym=zeros(1,geo.nelem);
geo.flap_vector=zeros(1,geo.nelem);

tor_version = 135;

%save the file into geo.mat such that it can be read by Tornado
save('geo.mat','geo','tor_version')

%% FINAL RESULTS

%0. For the number of panels specified, the span distribution is y
%1. The optimum chord distribution for number of panels specified -
%minimum twist with elliptic load
chord = c;
%2. The optimum relative twist for number of panels specified
rel_twist_deg = (alpha_tornado - AoA(end)+alpha0)*180/pi;
%3. Total Lift curve slope and zero angle of attack computation for the
%wing, not 2D airfoil
[a,CL0deg]=polyfit(AoA,CL3D,1);
```

```
%4. Required angle of attack for desired lift coefficient
requiredangleofattack=180/pi*(CL/(pi*AR))*(1+(8*s)/(c_root(i1,i2)*ao)) + alpha0*180/pi;
%5. Find L' distribution along span for the required AoA using LLT
index = find(abs(AoA*180/pi-requiredangleofattack)==min(abs(AoA*180/pi-requiredangleofattack)));
Sec_Lift = 1.225 * Uinf * gamma(index,:);
%Sectional lift for the best case
Ellipse = 1.225 * Uinf * 4*s*Uinf*A{index}(1,1).*sqrt(1-(y./s).^2);
%Mathematical elliptical load case

%Check that the sectional lift is indeed elliptic
figure
plot(y,Sec_Lift,'-^k')
hold on
plot(y,Ellipse,'r')

%Plot results for 3D wing
figure
set(gcf,'color','w')
plot(AoA*180/pi,CL3D)
title('change in total lift coefficient with AoA')
xlabel('\alpha (degree)')
ylabel('C_L')

figure
set(gcf,'color','w')
plot(AoA*180/pi,CDi)
title('change in induced drag coefficient with AoA')
xlabel('\alpha (degree)')
ylabel('C_Di')

figure
set(gcf,'color','w')
plot(AoA*180/pi,CL3D./CDi)
title('change in CL over CDi with AoA')
xlabel('\alpha (degree)')
ylabel('C_L/C_Di')

figure
set(gcf,'color','w')
plot(AoA*180/pi,L)
title('change in total lift with AoA')
xlabel('\alpha (degree)')
ylabel('Lift [N]')

figure
set(gcf,'color','w')
title('the ideal relative twist along the span')
plot(y_tornado,rel_twist_deg,'k','linewidth',3)
xlabel('Spanwise location [m]')
ylabel('Relative twist [degree]')
set(gca,'fontsize',30)
grid on

Cl_LLT = Cl(index,:);
save('LLT_CL3D.mat','AoA','CL3D') %AoA is in radians
save('LLT_Clspan.mat','y','Cl_LLT') %AoA is in radians


%% Create file for structures team (FATT04) with geometry and loading

struct.c_root = max(c);
struct.c_tip = min(c);
struct.g = g(i2);
struct.b = b;
struct.twist = rel_twist_deg;
struct.A = 1.225 * Uinf * 4*s*Uinf*A{index}(1,1);
struct.B = b/2;
struct.y = y_tornado;
struct.AR = AR;
struct.incidence = requiredangleofattack;
```

Appendix E: Validation Results for XFLR5 and Tornado

Both XFLR5 (3D Panel Code) and Tornado (VLM) were validated using the benchmark Warren 12 planform [29], which has the following geometry:

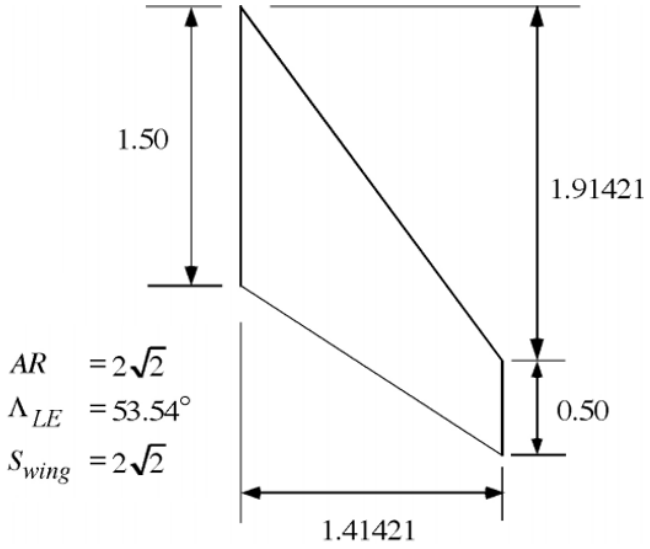


Figure 9.3: Warren 12 planform geometry

The values to compare are the lift curve slope, C_{L_α} and the pitching moment derivative with angle of attack, C_{M_α}.

	Warren 12	Tornado	Error	XFLR5	Error
C_{L_α} (/rads)	2.743	2.65	3.4%	2.93	6.8%
C_{M_α} (/rads)	-3.10	-3.08	0.65%	-3.08	0.65%

Table 9.1: Error estimation for Tornado and XFLR5 on the Warren 12 planform benchmark

Tornado presented a maximum error of 3.4% on the lift curve slope, which was deemed very satisfactory. XFLR5 had a maximum error of 6.8% on the same parameter and it was decided to carry a further test to validate this error estimate.
Using wind tunnel data from [30], the lift curve slope obtained from XFLR5 was compared to the experimental value. The following graph was obtained for the lift polars:

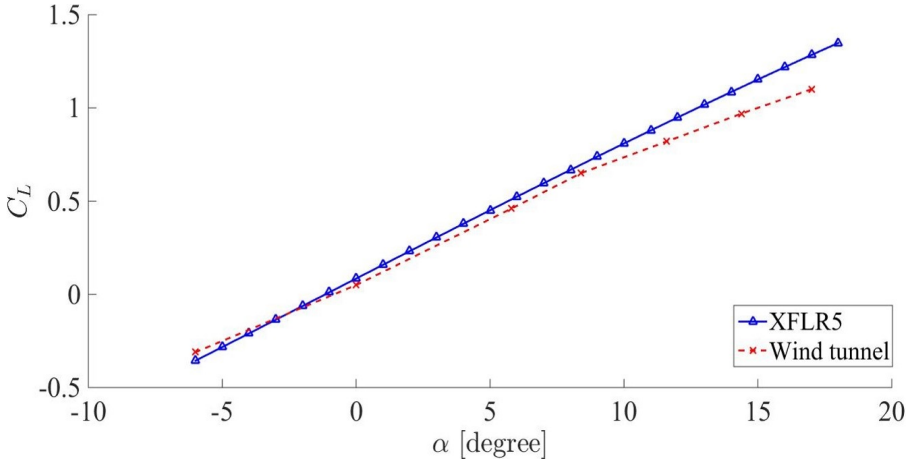


Figure 9.4: Lift polar comparison between XFLR5 and wind tunnel data from [30]

An error of about 7.2% was obtained between the lift curve slopes, which was deemed satisfactory due to XFLR5 being an inviscid method. Particularly, the C_L values at low angles of attack were found to be very accurate and thus validated the use of XFLR5 for these operating conditions.

Appendix F: Drag Build Up Method and Results

The method outlined in Raymer [3] is summarised here before presenting the results.
The zero lift drag coefficient is estimated in the following way:

$$C_{D_0} = \frac{\Sigma(C_f \, FF \, Q \, S_{wet})_{component}}{S} + C_{D_{leakage}} \tag{9.1}$$

where FF is a form factor that accounts for pressure drag from viscous separation, Q is an interference factor and C_f is the skin friction drag coefficient for a flat plate [3]. The latter can then be obtained as the flow was assumed to be fully turbulent:

$$C_{f_{turbulent}} = \frac{0.455}{(\log_{10} Re)^{2.58}} \tag{9.2}$$

where the Reynolds number used is the lowest of the one based on a characteristic length and the cut-off Reynolds number which accounts for surface roughness such as:

$$Re_{cut-off} = 38.21 \left(\frac{l}{k}\right)^{1.053} \tag{9.3}$$

where l is a characteristic length and k was taken as $k = 6096 \cdot 10^{-6} \, m$ for smooth paint from [3].
The form factors, FF, were calculated from the following formulae:
For the wing and tail:

$$FF = \left[1 + \frac{0.6}{(x/c)_{max}} \left(\frac{t}{c}\right) + 100 \left(\frac{t}{c}\right)^4\right] \left[1.34 \, M^{0.18}\right] \tag{9.4}$$

where $(x/c)_{max}$ is the position of maximum thickness, taken at $x/c = 0.3$. For the fuselage:

$$FF = \left[1 + \frac{60}{f^3} + \frac{f}{400}\right] \tag{9.5}$$

where f is the fineness ratio defined as $f = \frac{l}{d}$ where l is a characteristic length and d is the maximum diameter.
The interference factors Q were all taken as 1 due to the well-filleted wing except for the tail where Q was 1.04, typical for a conventional tail [3].
The drag due to leakage was estimated a 5% of the total drag due to the propeller [3]. The results for both the race and the application are presented in the table below.

Component	FF	Q	C_{D_0}	C_{D_i}	C_D
Wing	0.86	1	0.0090	0.0016	0.0106
Horizontal Tail	0.94	1.04	0.0017	$9.4 \cdot 10^{-5}$	0.0018
Vertical Tail	0.94	1.04	0.0008	-	0.0008
Fuselage	1.45	1	0.0010	-	0.0010
Total	-	-	0.0131	0.0017	0.015

Table 9.2: Race Conditions Drag Build Up

Component	FF	Q	C_{D_0}	C_{D_i}	C_D
Wing	0.86	1	0.0091	0.0105	0.0196
Horizontal Tail	0.94	1.04	0.0017	$9.4 \cdot 10^{-5}$	0.0018
Vertical Tail	0.94	1.04	0.0008	-	0.0008
Fuselage	1.45	1	0.0011	-	0.0011
Total	-	-	0.0134	0.0106	0.024

Table 9.3: Application Conditions Drag Build Up

Finally, the MATLAB code used to generate these drag values is shown below:

```
% Written by Jules Delpech and Lionel Kloetzli - May 2016
%-------------------------------------------------------------------------
% Drag build-up method calculations of cruise drag of the UAV "Solar
% Challenge". This method is based on D. Raymer, Aircraft Design:
% A Conceptual Approach, AIAA Education Series. 1989.
%-------------------------------------------------------------------------

clear
clc
close all

%% Inputs

m=3; %mass in kg
S_w=1; %wing area in m^2
S_th=0.1258; %tail area in m^2

c_mean_w=0.378; %wing mean chord in m
c_mean_th=0.1773; %horizontal tail mean chord in m
c_mean_tv=0.1732; %wvertical tail mean chord in m

l_f=0.47; %check with structure: fuselage length in m
d_f=0.091; %check with structure: fuselage diameter in m

l_N=0; %Nacell dimensions (if any)
d_N=0;

%Wetted areas, obtained from FATT02 (CAD)
Swet_w=1.895028;
Swet_f=0.142;
Swet_tv=0.123912;
Swet_th=0.259872;
Swet_N=0;

Aeff=0.0707; %windmilling: effective area of propeller
AR=7;
sweep_a_le=0;
AR_th=2/3*AR; %horizontal tail aspect ratio
sweep_a_le_th=0; %horizontal tail sweep angle
rho=1.225;
po=10.13e4;
nu=1.5e-5;

U_inf=16; %Velocity in m/s (16m/s for race and 10m/s for application)
M=U_inf/sqrt(1.4*287*(25+273.15)); %Mach number

CL=m*9.81/(0.5*rho*U_inf^2*S_w);
Cl_th=-0.1; %lift coefficient of the tail, given by FATT05

Re_cruise(1)=(rho*U_inf*c_mean_w)/nu; %for cruise
Re_cruise(2)=(rho*U_inf*c_mean_th)/nu;
Re_cruise(3)=(rho*U_inf*c_mean_tv)/nu;
Re_cruise(4)=(rho*U_inf*l_f)/nu;
%Re_cruise(5)=(rho*U_inf*l_N)/nu;

%% Estimation of Cf

%For laminar flow
Cflam_cruise=1.328./sqrt(Re_cruise);
%For turbulent flow (consider 100% turbulent flow)
per_lam=0;
per_tur=1;
per_lamt=0;
per_turt=1;
per_lamf=0;
per_turf=1;
%per_lamN=0;
%per_turN=1;

l_w=c_mean_w*3.28;  %feet mean chord wing
l_th=c_mean_th*3.28; %feet mean chord horizontal tail
l_tv=c_mean_tv*3.28; %feet mean chord vertical tail
%l_N=l_N*3.28;
l_f=l_f*3.28;     %feet fuselage
```

```
d_f=d_f*3.28; %feet diameter
k=2.08*10^(-5); %1.33 camouflage paint on aluminium (2.08 for smooth paint)

Re_co_cruise(1)=38.21*(l_w/k)^(1.053); %wing
Re_co_cruise(2)=38.21*(l_th/k)^(1.053);%horizontal tail
Re_co_cruise(3)=38.21*(l_tv/k)^(1.053);%vertical tail
Re_co_cruise(4)=38.21*(l_f/k)^(1.053);%fuselage
%Re_co_cruise(5)=38.21*(l_N/k)^(1.053);%nacelle

l_f=l_f/3.28;     %meter fuselage
d_f=d_f/3.28;     %meter diameter
%l_N=l_N/3.28;     %meter nacelle

for i=1:4
    if Re_cruise(i)<Re_co_cruise(i)
        Re_co_cruise(i)=Re_cruise(i);
    end
end

% cruise conditions
Cftur_cruise=0.455./((log10(Re_co_cruise)).^2.58); %skin friction turbulent wing
Cf_average_w(1)=per_lam*Cflam_cruise(1)+per_tur*Cftur_cruise(1);
Cf_average_th(1)=per_lamt*Cflam_cruise(2)+per_turt*Cftur_cruise(2);
Cf_average_tv(1)=per_lamt*Cflam_cruise(3)+per_turt*Cftur_cruise(3);
Cf_average_f(1)=per_lamf*Cflam_cruise(4)+per_turf*Cftur_cruise(4);
%Cf_average_N(1)=per_lamN*Cflam_cruise(5)+per_turN*Cftur_cruise(5);

%% Estimation of FF

x_c_max=0.3; %position of maximum thickness on wing
t_c=0.1;
t_c_th=0.1;
t_c_tv=0.1;

sweep_a_maxt=0; %Refers to the sweep of the maximum thickness line
sweep_a_maxt_th=0; %Refers to the sweep of the maximum thickness line
sweep_a_maxt_tv=0; %Refers to the sweep of the maximum thickness line

%Wing, tail, strut and pylon
FF_w=(1+(0.6/(x_c_max))*t_c+100*t_c^4)*(1.34*M^(0.18)*cosd(sweep_a_maxt)^0.28);
FF_th=1.1*((1+(0.6/(x_c_max))*t_c_th+100*t_c^4)*(1.34*M^(0.18)*cosd(sweep_a_maxt_th)^0.28));
FF_tv=1.1*((1+(0.6/(x_c_max))*t_c_tv+100*t_c^4)*(1.34*M^(0.18)*cosd(sweep_a_maxt_tv)^0.28));
%FF_N=1+0.35/(l_N/d_N);

%fuselage
f=l_f/d_f;
R=d_f/2; %max radius in meters
Amax=pi*R*R; %max croess-sectional area
FF_f=(1+(60/f^3)+(f/400));

%% Component interference factor Q

d=sqrt((4/pi)*Amax);
%Q_N=1;
Q_w=1; %consider the wing as well-filleted
Q_f=1;
Q_t=1.04; %1.04-1.05 for tail surfaces for a conventional tail

%% Induced drag

% Wing
e=1; %Perfect Oswald efficiency factor
K=1/(pi*AR*e);
Cdi=K*CL^2;

% Horizontal tail
e=1.78*(1-0.045*AR_th^0.68)-0.64; %from Raymer
k_f_t=1/(pi*AR_th*e);
Cdi_th=S_th/S_w*k_f_t*Cl_th^2;

%% Summation of every drag components

%Zero-lift drag coefficients
Cd0_wing=(Cf_average_w(1)*FF_w*Q_w*Swet_w)/S_w;
Cd0_th=(Cf_average_th(1)*FF_th*Q_t*Swet_th)/S_w;
Cd0_tv=(Cf_average_tv(1)*FF_tv*Q_t*Swet_tv)/S_w;
```

```
151 Cd0_f=(Cf_average_f(1)*FF_f*Q_f*Swet_f)/S_w;
152
153 Cdo_final_cruise=((Cf_average_w(1)*FF_w*Q_w*Swet_w)+(Cf_average_th(1)*FF_th*Q_t*Swet_th)+(Cf_average_tv(1)*FF_tv*
154
155 %Leakage drag is 5% of total drag
156 Cd_leakage=0.05*Cdo_final_cruise;
157 Cdo_final_cruise=Cdo_final_cruise+Cd_leakage;
158
159 %Total drag
160 Cd_cruise=Cdo_final_cruise+Cdi+Cdi_th;
161 Cdi_finale_cruise=Cdi+Cdi_th;
162 Cd_wing=(Cf_average_w(1)*FF_w*Q_w*Swet_w)/S_w+Cdi;
```

Appendix G: Empirical Transition Point Location Estimation

The e^N method used on XFoil with an N_{crit} of 13 was a first estimation of the transition point location, with the pressure distribution showing a kink at this point due to a laminar separation bubble being present [8]. This is shown in the following Figure where the C_P distribution is plotted over the chosen aerofoil, NACA 1810, at 1° angle of attack, corresponding to race conditions.

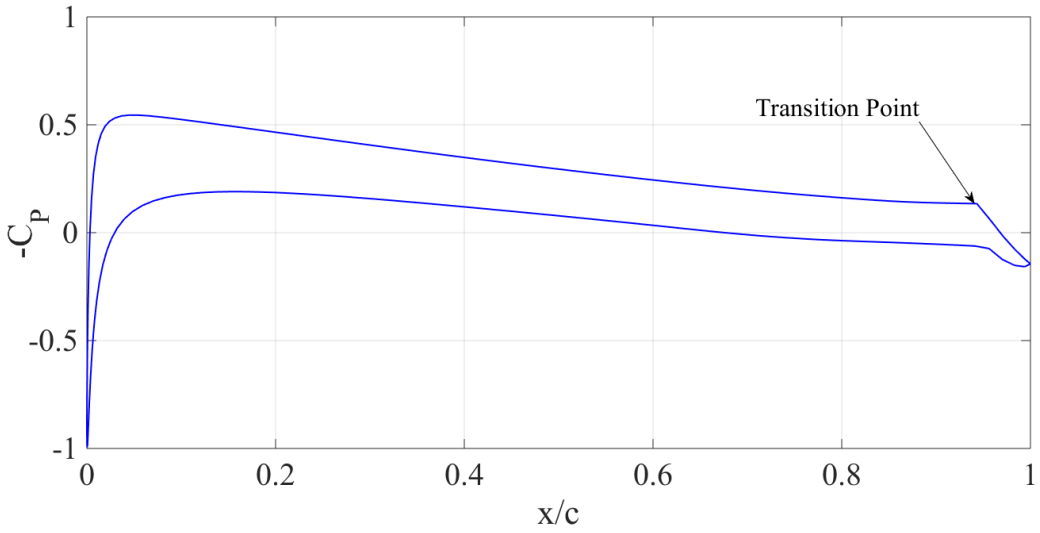


Figure 9.5: C_P distribution over a NACA 1810 at 1° angle of attack, as obtained from XFoil for an N_{crit} equal to 13

This estimates the transition point to be approximately at $(x/c)_{tr} = 95\%$.
Michel's method [20] [21] was also investigated. It suggests that transition occurs when the following is satisified:

$$Re_\theta \geq 1.174\,(Re_x^{0.46} + 22400\,Re_x^{-0.54}) \tag{9.6}$$

where Re_θ is based on the momentum thickness and on U_e which was determined from the pressure distribution obtained from XFoil. The momentum thickness, θ, was then calculated using Thwaites method such as:

$$\theta^2(x) = \frac{0.45\,\nu}{U_e(x)^6} \int_0^x U_e^5\,dx \tag{9.7}$$

Figure 9.6 shows a graphical interpretation of the inequality presented above.

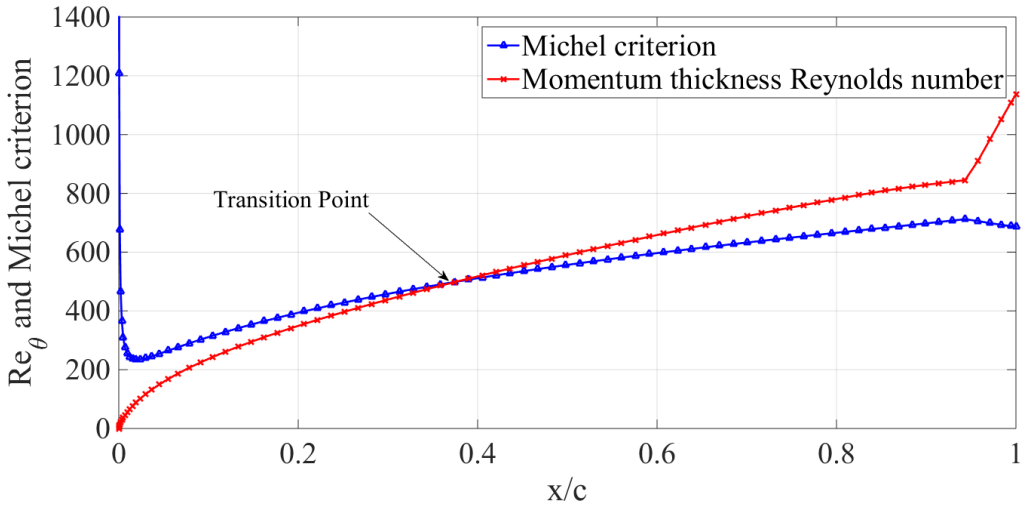


Figure 9.6: Estimation of the transition point location using Michel's criterion

This estimates the transition point to be approximately at $(x/c)_{tr} = 33\%$.

Appendix H: Fuselage Nose Cone Geometry Generation - Matlab Code

```
% Written by Lionel Kloetzli - 31/05/2016
%-------------------------------------------------------------------------
% Computes the fuselage nose shape according to J. Parsons, R. Goodsont,
% & F. Goldschmied, Shaping of Axisymmetric Bodies for Minimum Drag in
% Incompressible Flow, J. HYDRONAUTICS, VOL.8, NO. 3, 1974.
%-------------------------------------------------------------------------

clear
clc
close all

%% Inputs

D=0.09; %maximum diameter in metres
L=0.1; %length of nose in metres
Rn=0.01; %radius of curvature at the nose

%% Generate geometry

fr=L/D; %fineness ratio

K1=0; %curvature at Xm (0 for tangency with fuselage)
Xm=L; %position of junction to fuselage body
xm=Xm/L;
X=[0:0.00001:L];
x=X./Xm;

F1=-2.*x.*(x-1).^3;
F2=-x.^2.*(x-1).^2;
G=x.^2.*(3.*x.^2-8.*x+6);

rn=(4*fr)*Rn/L; %non dimensional curvature at the nose
k1=(-2*fr)*K1*L; %non dimensional curvature at the nose-fuselage body junction

r=L*(1/(2*fr)).*(rn.*F1+k1.*F2+G).^(1/2);
%Generate the radii distribution for the body or revolution

figure
hold all
plot(X,r,'k','linewidth',2)
plot(X,-r,'k','linewidth',2)
daspect([max(daspect)*[1 1] 1]);
set(gcf,'color','w')
set(0,'defaultaxesfontname','Times New Roman')
set(gca,'fontsize',30)
xlabel('Streamwise position [m]')
ylabel('Diameter [m]')
grid on
```

Appendix I: Determination of Separation Point Location and Boundary Layer Thickness

To determine the height of the vortex generators, the thickness of the boundary layer over the wing had to be estimated. The first occurrence of separation was observed using CFD at 10° incidence for the first iteration, and at 12° incidence for the second one. The flow was visualised on several 2D section of the wing to make sure that the separation point location was reliably estimated. A k-epsilon turbulence model was used as judged more accurate to visualise the flow [12]. The flow patterns for both iterations at the start of separation are shown in the Figures below:

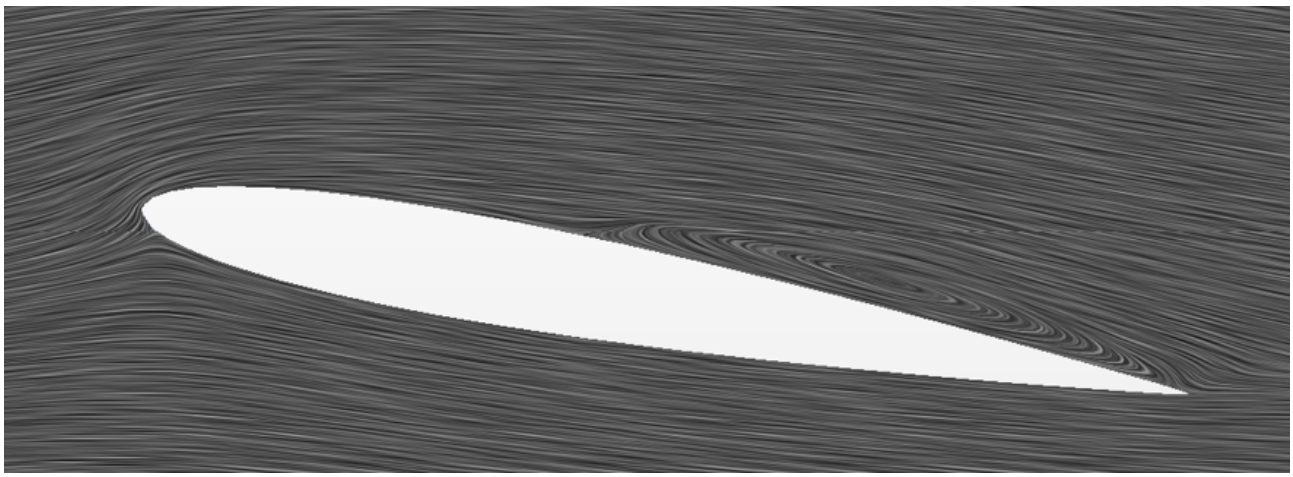


Figure 9.7: Onset of separation observed over the first iteration wing at 10° incidence

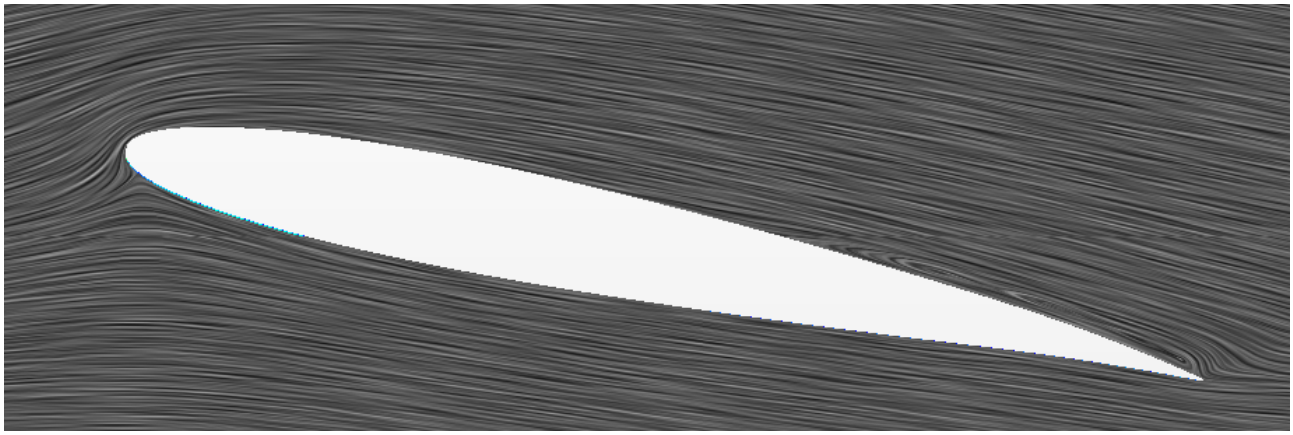


Figure 9.8: Onset of separation observed over the second iteration wing at 12° incidence

The boundary layer thickness was estimated using the following formula which is for a turbulent boundary layer developing over a flat plate [26]:

$$\delta(x) = \frac{0.385\,x}{Re_x^{1/5}} \tag{9.8}$$

where Re_x is defined as:

$$Re_x = \frac{U_e\,x}{\nu} \tag{9.9}$$

and for which U_e was obtained from the C_P distribution from XFoil for the NACA 1810 aerofoil at 12°, as:

$$U_e = U_\infty \sqrt{1 - C_P} \tag{9.10}$$

Appendix J: Vortex Generator Geometry

The vortex generators mentioned in this report are shown in the following Figures.

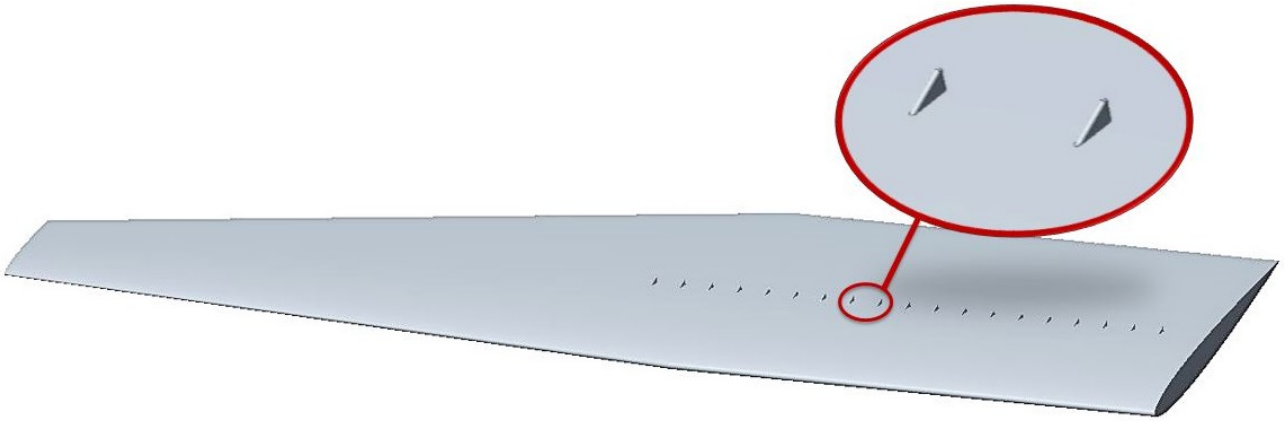


Figure 9.9: Co-rotating vortex generators studied in this report (iteration 1)

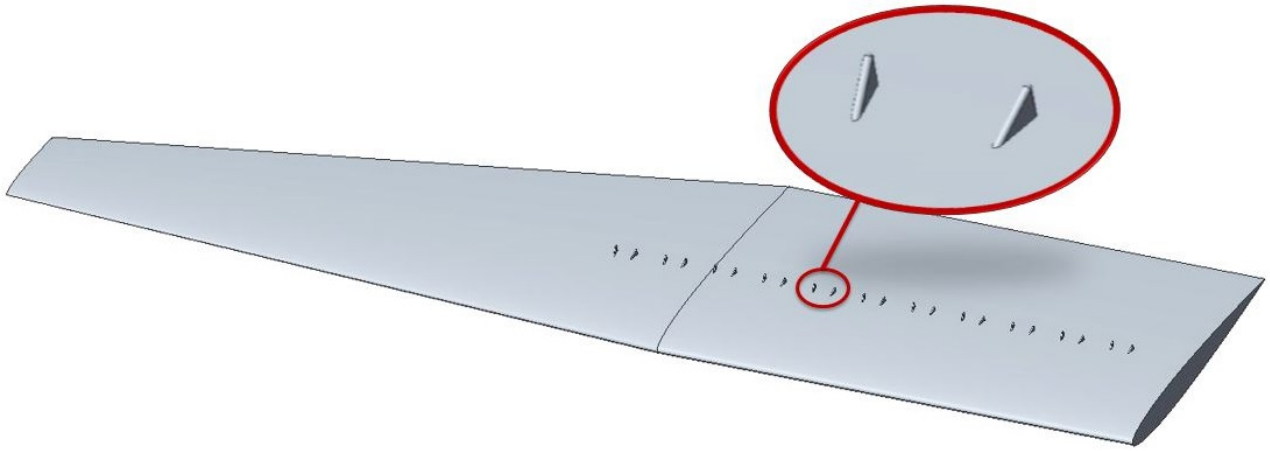


Figure 9.10: Counter-rotating vortex generators studied in this report (iteration 1)

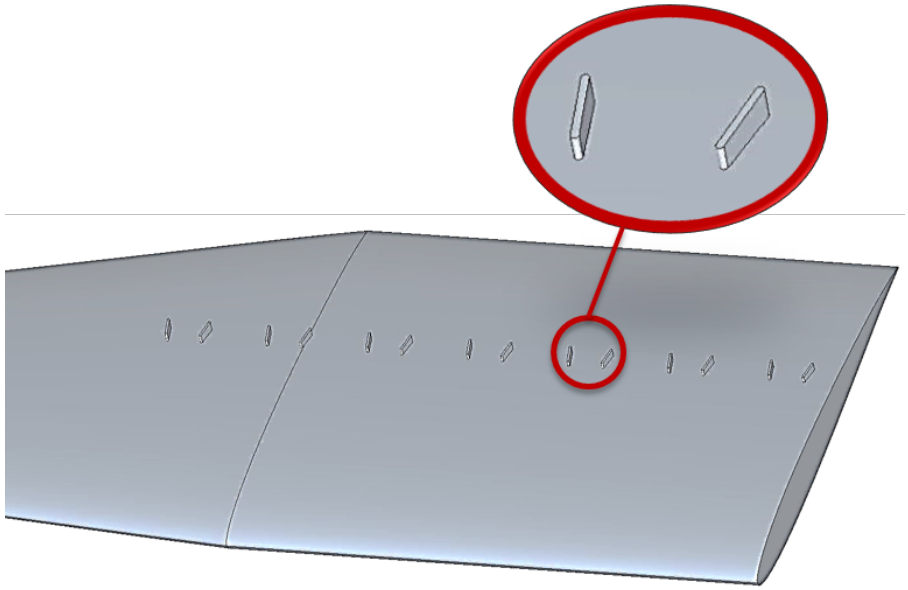


Figure 9.11: Selected optimal vortex generator geometry [11] placed on the second iteration wing

Table 9.4 shows the main dimensions of the overall most efficient vortex generators from [11] and the optimum ones as obtained in this report.

	Overall optimum geometry [11]	Optimum geometry as obtained in this report
Characteristic	Rectangular, Flat plate	Tapered, Flat plate
Type	Counter Rotating	Counter Rotating
Spacing between each set	100 mm	56 mm
Spacing between each VG	32 mm	34 mm
Thickness	2 mm	2 mm
Length	20 mm	14 mm
Angle	10 degrees	10 degrees
Height	8 mm	8 mm

Table 9.4: Summary of vortex generators geometries

Appendix K: Wing Tip Flow Visualisation Using CFD

The flow was visualised using streamlines on Star-CCM+ for the second iteration wing with and without the aforementioned wing tip geometry at the angles of attack and velocities corresponding to race and application conditions.

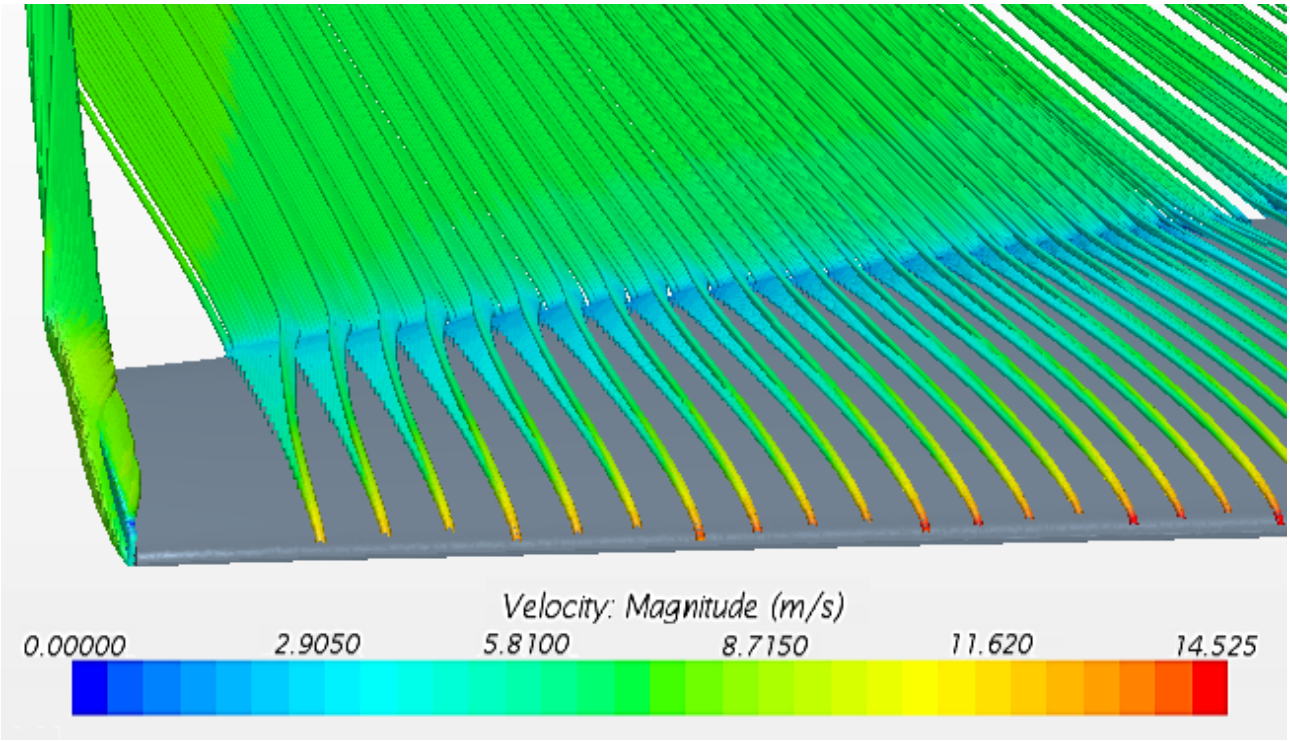


Figure 9.12: Star-CCM+ flow visualisation of streamlines around the wing with no added wing tip

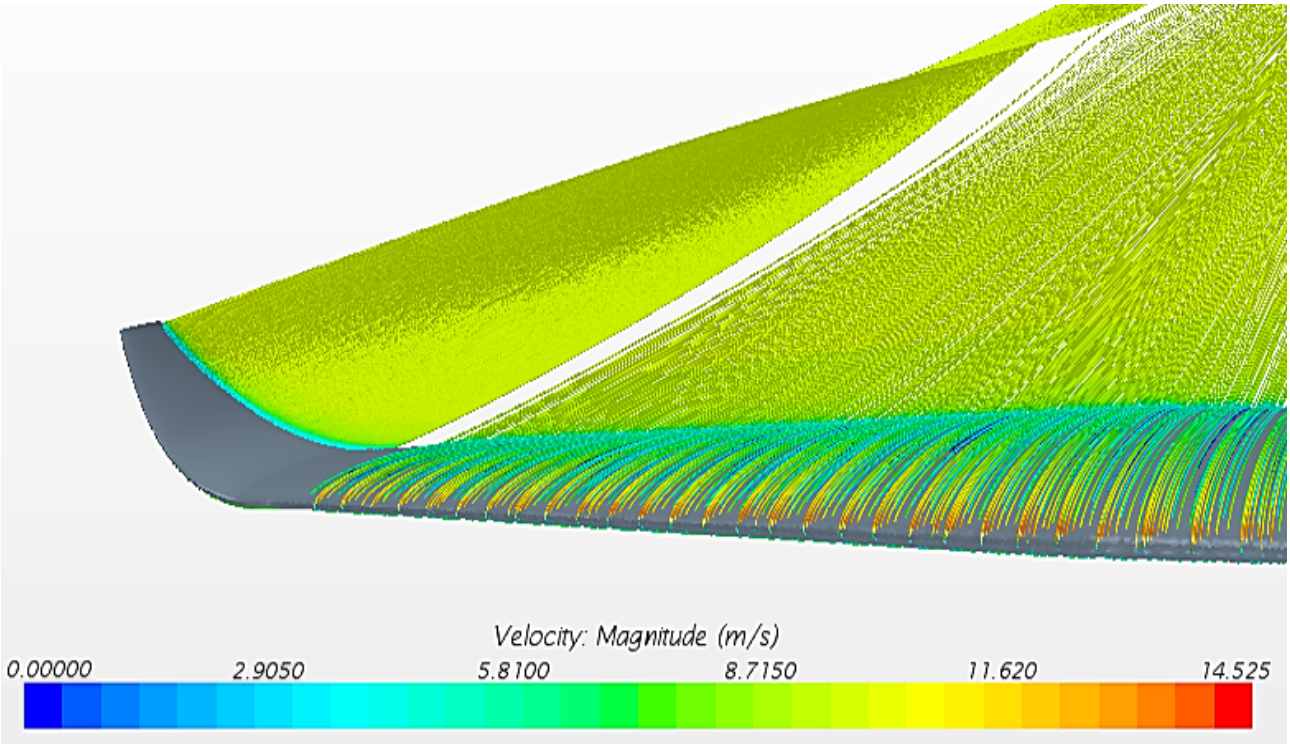


Figure 9.13: Star-CCM+ flow visualisation of streamlines around the wing with the added wing tip